

Dynamic Resource Management for Intelligent Transportation System Applications

S M Abdulla Al Mamun

A dissertation submitted to The University of Manchester
for the degree of Master of Science in Artificial Intelligence
in the Faculty of Science and Engineering

Year of submission 2026

Department of Computer Science, School of Engineering

Supervisor: Dr Sandra Sampaio

Contents

Abbreviations	5
Glossary	6
Abstract	7
Declaration	8
Copyright statement	9
Acknowledgements	9
1 Introduction	10
1.1 Problem and motivation	10
1.2 Related work and the position of this study	12
1.3 Aim, objectives and research questions	14
1.4 Scope and report structure	15
2 Methodology	15
2.1 Research design and evidence hierarchy	15
2.2 Architecture and information boundaries	16
2.3 Scenarios, actor and controlled inputs	16
2.4 Time, queues, admission and outcomes	18
2.5 Placement algorithms and their implementation	20
2.6 Experimental progression and inference	22
2.7 Sensitivities, mechanism audit and validation	23
2.8 Cyclic comparator and retrospective measurement design	24
2.9 Method selection and alternatives	24
3 Evaluation and Reflection	25
3.1 Validation and scoping: accounting and historical capacity	25
3.2 From admission decomposition to the incident reversal	25
3.3 Morning replication with the inspected pilot excluded	27
3.4 Confirmation under joint randomness and comparison with cyclic spreading	29
3.5 Why destination semantics matter	32
3.6 Bounded state-information and forwarding sensitivities	33
3.7 Computational trade-offs and research implementation	33

4 Conclusion	37
4.1 Analysis of the project approach	38
4.2 Research answers and interpretation	38
4.3 Prioritised future work	39
4.4 Reflection on the process	39
References	39
Appendix A. Evidence sources and reproducibility	43
Appendix B. Audit coverage and diagnostic boundaries	46
Appendix C. Exact-model result and numerical boundary	49
C.1 Model and objects that must remain separate	49
C.2 Existence, uniqueness and candidate-count bound	49
C.3 A deadline-count bound from eligibility	50
C.4 Why float32 requires a separate claim	50
C.5 Consequence for queue clearing in the stated exact model	51
C.6 Retained diagnostic evidence	51
C.7 Retained adverse dispatch illustration	52
C.8 Reconstruction and task-outcome details	53
Appendix D. Type aggregation and engineering evidence	56
Appendix E. Joint-randomness study controls and execution	58
Appendix F. Latency, scoring and drain details	61
Appendix G. Artefact access and verification	62

Word count: 8,386

List of Figures

1	Experimental responsibility boundaries	11
2	Morning RSU coordinates	17
3	Offered-task lifecycle	19
4	Illustrative dispatch comparison	22
5	Four-draw morning replication	28
6	All eight joint-randomness blocks	30
7	TrafficTwin with real data: external package	35
8	TrafficTwin with real data: dissertation confirmation	36
9	Morning RSU work distribution	53

List of Tables

1	Closest-work comparison	13
2	Scenario identities and controls	17
3	Infrastructure conditions	20
4	Study map	22
5	Method selection and alternatives	24
6	Incident attainment	27
7	Morning primary replication	28
8	Morning paired contrasts	28
9	Eight-block simultaneous contrasts	29
10	Declared follow-up contrasts	31
11	Three-trace simultaneous contrasts	32
12	Validity limits	37
A1	Execution environment	44
A2	Software components and checks	45
B1	Historical audit coverage	46
B2	Matched task transitions	47
C1	Adverse dispatch illustration	52
C2	Morning outcome accounting	54
C3	Type-level net changes	55
D1	Type outcomes	56
D2	Scheduler-only engineering trade-offs	57
E1	All confirmation cells	59
E2	Eight paired effects	60

Abbreviations

CI	Confidence interval
CLI	Command-line interface
CPU	Central processing unit
DRL	Deep reinforcement learning
E0–E2d	Historical validation, scoping and dispatch study stages
EV	Electric vehicle
ITS	Intelligent transportation systems
JAX	Array-computing and automatic-differentiation framework
MAPPO	Multi-agent proximal policy optimisation
PPO	Proximal policy optimisation
RQ	Research question
RSU	Roadside unit
SoC	State of charge
SUMO	Simulation of Urban MObility
V2I / V2V	Vehicle-to-infrastructure / vehicle-to-vehicle
VEC	Vehicular edge computing

Glossary

- Offered task** A generated task presented to the system remains in the primary outcome population whether admitted, rejected or unavailable. [Section 2.4]
- Admission** Acceptance into execution accounting and a queue; an admitted task can still miss its deadline.
- Non-admission** Rejection by an admission check or unavailability of the selected route/resource; no work is reserved. [Section 2.4]
- Backlog gate** The strict gate compares existing effective service workload with the candidate deadline, excluding its own service and transmission time. [Section 2.4]
- Task-count capacity** The queue limit constrains the number of tasks and does not increase computing service. [Section 2.4]
- Ingress RSU and ingress execution** Ingress is the strongest-link roadside entry point, and ingress execution keeps the task at that server. [Section 2.2]
- Common-target placement** One least-workload destination is selected per substep and shared by its V2I candidates. [Section 2.5]
- Causal per-task placement** Each task selects a destination using workloads updated after earlier admissions. Here causal means ordered decisions, not causal inference. [Section 2.5]
- Causal round-robin** The cyclic comparator spreads candidates while retaining causal admission and immediate reservation. [Section 2.8]
- Two-choice (d1a_p2c)** The gate-enabled alternative selects the lower-workload RSU from two samples per vehicle, drawn with replacement. [Section 2.5]
- Substep** An internal task slot orders admissions within a batch without advancing the simulation clock. [Section 2.4]
- Drain** After the final substep, queued service work is reduced by the batch service allowance. [Section 2.4]
- Fleet draw and fleet seed** The fleet seed selects the draw of vehicle tier, electric-vehicle status and initial state of charge. [Section 3.4]
- Evaluator seed** This seed varies observations, arrivals, task types and sizes, fading and service wobble. [Section 3.4]
- Joint block** A paired fleet/evaluator-seed block supplies matched runs across the compared policies. [Section 2.6]
- Cell** A cell is a full evaluator run for one policy under its specified seed block. [Section 3.4]
- Sealed protocol** Controls and analysis are fixed before the full study outcomes are observed. [Section 2.6]
- Receipt** A validation record binds a completed run to its source, seeds and outputs. [Section 3.7]
- Percentage point (pp)** Paired differences subtract one policy's attainment percentage from another's on the same draw. [Section 2.6]
- Attainment** Offered-task deadline attainment is the share of all offered tasks recorded as meeting their deadlines. [Section 2.4]

Abstract

TrafficTwin is a research-software platform for configuring, comparing and inspecting vehicular edge-computing studies. This dissertation uses a supplied trace-driven evaluator to ask how infrastructure scheduling affects the proportion of offered tasks meeting their deadlines under a fixed pretrained vehicle policy. Radio ingress, execution placement and admission are treated as separate decisions. Seven infrastructure policies were compared in 274 full evaluator runs across five Manchester scenarios.

Common-target least-workload placement underperformed execution at ingress, whereas sequential per-task placement reversed this ranking in incident and morning comparisons. In a sealed confirmation with eight fresh paired fleet/evaluator-seed blocks and four policies, per-task placement improved attainment over ingress by 4.137 percentage points. It also exceeded causal round-robin by 0.631 points, with a Bonferroni simultaneous 95% interval of [0.511, 0.750]. Thus workload-aware selection added a smaller benefit beyond cyclic spreading.

Declared follow-ups found an advantage over two-choice placement, a larger per-task advantage over round-robin at half server speed, and the same ranking with a second actor checkpoint. The ranking also recurred on weekend, evening-peak and event-night traces.

The contribution is a controlled comparison and explanation of how destination selection and workload reservation affect deadline attainment in this evaluator. The findings are limited to the tested model and scenarios from one Manchester network; they do not establish deployed-system performance.

Declaration

No portion of the work referred to in the dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Assistance and attribution. I selected the research questions and studies, directed implementation, and take responsibility for the analysis and interpretation. Following literature review and brainstorming with large language models (LLMs), and subsequent discussion with my PI and permission to proceed, I selected the follow-up conditions and three-trace study and directed their implementation. Codex and ChatGPT assisted with code, campaign tooling and manuscript drafting and revision; Claude assisted with discussion of alternatives and draft critique. Protocols were sealed before their respective outcomes. Randy Prasetia Putra supplied the MAPPO actor, the `vec_env` environment and evaluator, and implemented the 5 August accounting repair. Dr Sandra Sampaio suggested the report-age levels in Section 2.7. Section 3.7 identifies the recorded verification exercises.

Signature: _____ Date: _____

Copyright statement

- (i) The author of this dissertation (including any appendices and/or schedules to this dissertation) owns certain copyright or related rights in it (the “Copyright”) and they have given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- (ii) Copies of this dissertation, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has entered into. This page must form part of any such copies made.
- (iii) The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the dissertation, for example graphs and tables (“Reproductions”), which may be described in this dissertation, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- (iv) Further information on the conditions under which disclosure, publication and commercialisation of this dissertation, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy, in any relevant Dissertation restriction declarations deposited in the University Library, and The University Library’s regulations.

Acknowledgements

I thank Dr Sandra Sampaio for supervising this project and suggesting the report-age levels. I thank Randy Prasetya Putra for supplying the MAPPO actor, the vec_env environment and evaluator, and the accounting repair.

1 Introduction

1.1 Problem and motivation

Vehicular edge computing concerns where a vehicle’s computational tasks should run and whether their results can return before a deadline. A task may execute locally, use a neighbouring vehicle through vehicle-to-vehicle communication (V2V), or enter roadside infrastructure through vehicle-to-infrastructure communication (V2I) [1], [2]. A roadside unit (RSU) combines a radio access point with a simulated computing server in this study. Offloading therefore couples communication conditions with the work already waiting for computation [3].

WHO’s *Global status report on road safety 2023* estimates 1.19 million road traffic deaths in 2021 [4], motivating research into timely information sharing. 3GPP TS 22.186 v16.2.0 sets maximum end-to-end latencies of 100 ms for automated-driving information sharing between a vehicle and an RSU (clause 5.3, Table 5.3-1) and 500 ms for platooning reporting, including vehicle–RSU communication (clause 5.2, Table 5.2-1) [5]. The study adopts 100 ms and 500 ms as task-completion deadlines motivated by those two communication requirements; treating them as deadlines for the whole offloaded task is a modelling choice.

A result must arrive within the application’s usable time window to count as timely [6]. Offloading provides additional computing resources, but transmission and waiting consume that window. The mobile-edge literature therefore treats communication and computation jointly [3]. This study measures timely results over all offered tasks, including work rejected before execution.

Mobility makes this harder. Vehicular radio channels vary quickly [7]. Meanwhile, the vehicles around each RSU change while previously admitted work remains queued, producing uneven server loads [8]. Vehicular edge-computing research therefore considers communication and resource allocation jointly under latency requirements [9].

The scenarios come from a count-matched, demand-scaled Manchester SUMO network around Etihad and Co-op Live [10]. I use saved SUMO working-day and gridlock traces as repeatable inputs, exposing the scheduling rules to different conditions in the same intelligent-transport setting. The infrastructure decision is whether to retain ingress execution, distribute work or alter admission, subject to available service and coordination costs.

Research on RSU-to-RSU cooperation [8] and task inference across vehicle, RSU and edge resources [11] supplies a wider context for execution placement, as does deep-reinforcement-learning offloading in vehicular edge networks [12], [13], [14].

TrafficTwin is the research-software platform developed for this project around a supplied trace-driven VEC evaluator. Its inherited `vec_env` environment selects the **ingress RSU**, the roadside entry point, using the strongest simulated radio link. **Execution placement** then selects the server that will process an admitted task. Keeping execution at ingress avoids forwarding, but strong radio quality does not imply a short server backlog. Figure 1 distinguishes actor mode selection, environmental radio targeting, and infrastructure placement and admission [S4].

Admission is distinct from timely completion. In addition to a task-count limit, the inherited evaluator rejects a candidate when service work already ahead of it is at least its deadline. This backlog-only gate excludes the task’s own processing and communication, so admitted tasks may still finish late. All offered tasks remain in the outcome denominator, preventing selective rejection from hiding unsuccessful work.

To study these infrastructure decisions, I froze the weights of the supplied multi-agent proximal policy optimisation (MAPPO) actor. Communication and computation jointly determine whether an offloaded task returns in time [3], [9]. Matched inputs and action checks distinguish infrastructure effects from retraining and possible observation feedback. The comparison therefore concerns scheduling implementations that support an existing actor [S0]–[S4].

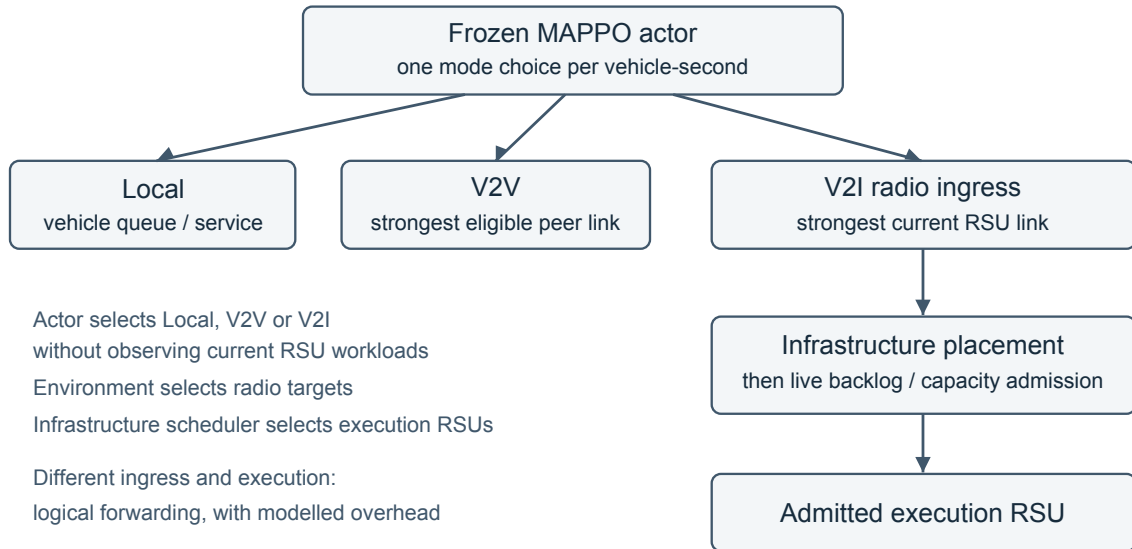


Figure 1: Experimental responsibility boundaries. Only admitted V2I work enters the selected execution queue; logical forwarding applies when ingress and execution differ. Resource scaling is outside the completed comparison [S11].

1.2 Related work and the position of this study

Because the vehicle policy is fixed, the closest scheduling comparison is parallel-server dispatch. The relevant questions are what information represents waiting work and when each assignment updates that information.

Queue count and remaining work. Shortest-queue and shortest-workload rules compare job counts and outstanding service respectively [15], [16]; Harchol-Balter, Crovella and Murta study immediate per-arrival assignment with known service demand and identical hosts [17]. The dependence of the preferred policy on task-size variability motivates distinguishing TrafficTwin’s service-millisecond selector, historically named `jsq`, from its separate task-count admission limit.

Batch decisions and reservations. Sparrow spreads parallel-job tasks over sampled workers and binds queued reservations when workers become ready, addressing queue-count errors and scheduling races [18]. Its explicit reservations highlight two separate TrafficTwin choices: whether candidates share a destination and when newly admitted service work becomes visible to subsequent decisions.

Ying, Srikant and Kang make the reservation distinction especially explicit: batch-filling samples queue counts, assigns tasks one by one and updates the chosen queue after each assignment. Their analysis assumes identical servers, exponential service, Poisson batch arrivals and random ties [19]. This is close prior art for sequential within-batch updates. The evaluator also implements a power-of-two-choices variant (`dla_p2c`) that was evaluated in the declared follow-ups (Section 3.4) [20]. TrafficTwin’s causal change is therefore an adaptation of established dispatch semantics, evaluated with workload information and deadline admission.

Local information and common destinations. Vargaftik, Keslassy and Orda’s Local Shortest Queue policies route dispatcher batches using possibly outdated queue counts, local increments and communicated updates under stated stability assumptions [21]. This work, together with studies of stale-information herding and local updates [22], [23], motivates specifying TrafficTwin’s service-work information, deterministic ties and aggregate draining when interpreting finite-horizon deadline outcomes.

Admission and useful completion. Niño-Mora jointly models admission and routing with separate costs for rejection and admitted deadline misses under Poisson arrivals, exponential service and soft deadlines [24]. This motivates TrafficTwin’s offered-task denominator, which includes rejections alongside admitted misses when evaluating its inherited backlog filter.

Kovalenko et al. report lower deadline-miss rates with heterogeneous federation than no redirection [25]. Their probabilistic placement and communication model differ from TrafficTwin’s frozen-actor comparison; Table 1 compares mechanisms.

Table 1: Closest-work comparison. “Work” means remaining service demand; “count” means queued jobs. TrafficTwin rows describe the frozen implementation. Scheduling labels alone do not imply equivalent decision rules.

Study / policy and assignment unit	Queue information and updates	Admission / objective	Relationship to TrafficTwin
Harchol-Balter et al. [17]: dispatch each arrival	Known service work; central per-arrival choice	Mean waiting time; size-normalised waiting	Closest least-workload comparator; no vehicular gate
Sparrow [18]: parallel-job task placement	Sampled workers; queued reservations; worker replies bind tasks	Job response time; placement constraints	Batch spreading differs from one common destination; communication is explicit
Ying et al. [19]: tasks within an arriving batch	Sampled counts; update after each assignment; random ties	Delay and sampling cost	Closest within-batch update rule; no deadline gate
LSQ [21]: dispatcher batch per time slot	Local queue counts; own-assignment increments and sampled/pushed updates	Stability under specified assumptions	Common routing is established; random ties and distributed estimates differ
Niño-Mora [24]: joint admission/routing per arrival	Queue counts and model-based indices	Rejection and deadline-miss costs	Supports joint evaluation, not the inherited backlog predicate
<i>Implementations evaluated in this dissertation</i>			
TrafficTwin common-target: one choice per substep	Global service work; vectorised candidate offsets and three reconciliations	Backlog gate plus finite task limit	Frozen vehicle policy; instantaneous logical forwarding
TrafficTwin per-task: ordered candidate scan	Global service work; actual admitted work reserved immediately	Same predicate and limit, causal admission	Tested implementation change; global information is assumed

Simulation validity. Conservation and replay checks verify implementation of the stated model; validation for deployment would additionally require comparison with measured system behaviour [26]. SUMO supplies the traffic simulation framework [27], but its trajectories alone do not validate the coupled radio/computing model.

PPO and MAPPO identify the reused actor’s provenance [28], [29]. Henderson et al., Agarwal et al. and Gorsane et al. motivate exposing implementation choices and finite-run uncertainty [30], [31], [32]. TrafficTwin checks action equality empirically and uses draws or joint-seed blocks, not tasks, as replications.

Digital-twin work spans distinct interventions [33]. Dasgupta et al. study adaptive traffic control [34]; Xie, Wu and Fan study offloading and allocation with estimation error at a single base station [35]. Cho et al. explicitly model parallel computation queues [36].

These differ from the fixed-actor, multiple-RSU dispatch comparison here, motivating clear control and queue semantics.

Communication remains part of feasibility: Wu et al. model heterogeneous links under latency/reliability constraints [9], while hybrid vehicular/cloud offloading coordinates communication-constrained V2V and edge resources [37]. TrafficTwin instead fixes the vehicle actor's weights to study infrastructure dispatch.

1.3 Aim, objectives and research questions

The aim is to assess how infrastructure admission and placement affect deadline attainment under a frozen vehicle actor. Five objectives organise the programme.

O1 — Establish the measurement contract. Distinguish offers, admissions, failures and deadline success. Qualify the contract through accounting validation (E0) and capacity scoping (E1), retaining historical evidence limits.

O2 — Distinguish dispatch alternatives. Specify ingress, common-target, causal per-task and cyclic placement, including control boundaries, reservation timing and validation.

O3 — Estimate matched contrasts. Compare incident and morning draws, then test declared contrasts on separate joint-randomness blocks.

O4 — Explain mechanisms and boundaries. Connect dispatch to destination concentration, the conditional exact result and completed information-age/forwarding sensitivities.

O5 — Deliver an inspectable artefact. Document evaluator, campaign, verification and product components with reproducible inventory and evaluation boundaries.

RQ1: Under matched gate-enabled controls, how do common-target and per-task placement compare with ingress execution, and what explains the direction reversal? **RQ2:** Does per-task placement improve offered-task deadline attainment over round-robin and two-choice placement? Does the ranking recur across the tested traces, at half RSU service rate and with a second saved actor checkpoint? **RQ3:** What boundaries are established by the completed information-age and fixed-forwarding sensitivities?

Established work addresses least-workload routing, within-batch updates and vehicular cooperation [8], [11], [17], [19]. This study examines whether destination selection and reservation timing reverse a matched deadline-attainment comparison under fixed actor weights. Section 4 relates the outcomes to the objectives.

Contributions. The Declaration distinguishes supplied components and assistance. My contributions are:

C1. Identifying the scoring/admission mismatch in the supplied evaluator; Putra implemented the repair, with a remaining historical limitation documented in Section 3.1.

C2. Explaining common-target concentration through destination timing, deterministic ties, the $\min(K, R)$ bound and Proposition 1.

C3. Specifying and implementing per-task and round-robin placement with explicit reservation, restart and pointer rules.

C4. Designing the sealed runner, validators and analysis for 32 valid confirmation cells and eight block receipts without retries.

C5. Designing and executing matched policy comparisons, from 82 initial full evaluator runs to 274 with the declared follow-ups and three-trace study.

C6. Completing report-age/forwarding sensitivities, scheduler benchmarks and the product inventory, with their recorded limitations.

1.4 Scope and report structure

The completed research evaluates the trace-driven VEC scheduler; Section 3.7 describes the implemented TrafficTwin workflows and their evaluation boundaries.

A user evaluation was not completed, so usability and operator benefit remain unmeasured. The separate Dynamic Resource V2 (E3) research campaign was unexecuted. Section 2 specifies the methods, Section 3 evaluates completed work, and Section 4 concludes [S11]. The registered brief [38] is addressed through the methodology, task classes and scenarios, scheduler runtime benchmark, and resource-management comparisons; these do not constitute a full application-runtime or operator evaluation.

Ethical considerations. This study used simulated SUMO traces and involved no human participants or personal data. My University of Manchester Ethics Decision Tool check on 15 September 2026 indicated that ethics approval was not required [39].

2 Methodology

2.1 Research design and evidence hierarchy

The programme progressed from accounting/capacity checks and exploratory comparisons to adaptive per-task placement, prospective replication, sealed confirmation and declared follow-ups (Table 4). Matched fleet draws or joint fleet/evaluator-seed blocks supply the paired replications; later studies adjust intervals within declared contrast families [S0]–[S8], [S13].

2.2 Architecture and information boundaries

Figure 1 separates actor mode choice, environmental radio targeting and infrastructure placement and admission. Ingress execution processes a task at the same RSU that receives its V2I transmission; other policies may assign computation elsewhere. Every target remains a proposal until admission [S4].

The 17-dimensional observation includes task descriptors, vehicle queues, battery state of charge (SoC), radio summaries, compute capability, EV status and observed V2V target tier, but excludes RSU workload. The actor chooses one offloading mode per active vehicle per one-second batch; all tasks generated for that vehicle in its up to five slots use that mode. Admission can affect transmit energy, then SoC and later observations. Fixed weights therefore do not guarantee identical actions; the older action-matching finding is empirical, and the later protocol allows this feedback.

V2V helper selection follows the inherited strongest-link convention, held constant across arms (Appendix F).

2.3 Scenarios, actor and controlled inputs

The initial two scenarios are drawn from the Manchester SUMO working-day and gridlock traces: `manchester_workingday/trace_wd_am_wdrsu.npz` and the 15 March 2024 gridlock trace (Table 2). Both traces use Putra et al.’s count-matched, demand-scaled Manchester SUMO network around Etihad Stadium and Co-op Live [10]. The source study builds the network from OpenStreetMap, scales demand uniformly to 70% of observed sensor counts to avoid simulated gridlock, and reports SUMO 1.27.0 [10]. The incident hour is that study’s documented reactive-control collapse on 15 March 2024, used here as a simulated dense-traffic stress case rather than a recorded incident. Coordinates are local trace metres. Evaluations import saved positions without rerunning SUMO.

Morning input checks reconstruct canonical arrays and verify entry markers because padded slots need not retain one vehicle identity. Queues reset for each new visit; SoC does not. The incident trace retains its earlier convention without the same entry channel [S7].

Figure 2 plots the nine simulated RSU positions stored in the morning trace’s `rsu_xy` array, using local SUMO metres. Incident coordinates are absent from the compact evidence. Each comparison preserves its supplied trace and layout (Table 2).

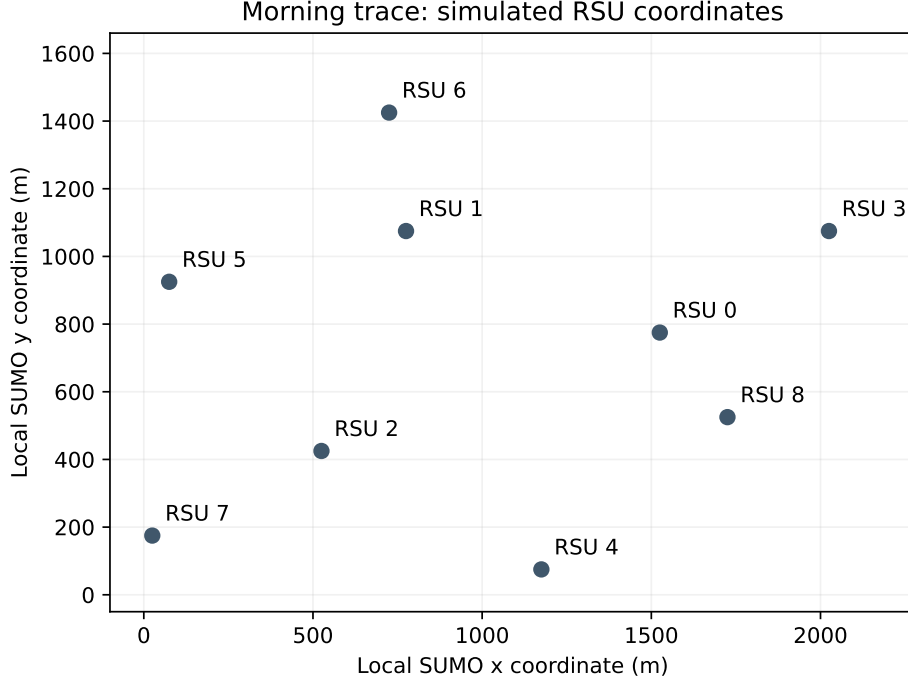


Figure 2: The nine simulated RSU positions from the morning trace’s `rsu_xy` array, in local SUMO metres with zero-based indices. The nine locations remain fixed while the dispatch comparisons change task destinations [S8].

Table 2: Scenario identity and fixed placement controls; the incident window is a simulated network-lockdown hour from [10]. Differences between scenarios are retained and documented, not treated as a single-factor intervention.

Setting	Incident scenario	Morning scenario
Date and local window	15 March 2024, 20:00–21:00	15 October 2024, 08:00–11:00
One-second steps	3,600	10,800
Padded vehicle slots	2,488	215
RSUs	10	9
SUMO version (source study)	1.27.0 [10]	1.27.0 [10]
SUMO seed	43	42
Vehicle queue convention	Conserved, legacy trace convention	Conserved, canonical per-visit reset
Placement-study RSU limit	6,220 tasks per RSU	6,220 tasks per RSU
Service / forwarding / scaling	1× / 0 ms / off	1× / 0 ms / off

An archived MAPPO checkpoint (a saved set of actor weights) and device-tier preset are fixed in the initial studies. Section 3.4 tests a second checkpoint sharing the training seed.

For each active vehicle and second, draw $X \sim \text{Poisson}(1.5)$ and offer $K = \min(X, 5)$ tasks; inactive slots offer none. Operational types are independently sampled with probabilities (0.20, 0.30, 0.50). Types 1–3 have deadlines (100, 500, 100) ms, mean input sizes (1, 0.0012, 0.001) MB and workloads (1,254, 2,100, 15) million cycles. Sizes multiply the

type mean by an independent uniform 0.8–1.2 factor. The Poisson mean is before capping; the cap changes offered work as well as scheduling opportunities [S4].

At nominal RSU service multiplier 1, task cycles, frequency, parallelism, utilisation and acceleration give approximately 21.1, 35.4 and 2.53 ms for types 1–3, before the 0.9–1.1 noise multiplier (Appendix F).

Absolute capacity avoids a confound: $2.5 \times 2,488 = 6,220$ tasks per RSU in the incident scenario. Applying that ratio to 215 morning slots gives 537.5, rounded by the evaluator to 538. The morning protocol instead fixes 6,220; density, duration, layout and entry conventions still differ.

2.4 Time, queues, admission and outcomes

A batch advances simulated time by one second. Five task substeps order admissions without advancing the clock. At substep entry, $W[r]$ is remaining service work at RSU r in milliseconds, $Q[r]$ its aggregate task count and C its task-count limit. Equal counts can represent different workloads. Admissions accumulate across the five substeps before one service drain [S4].

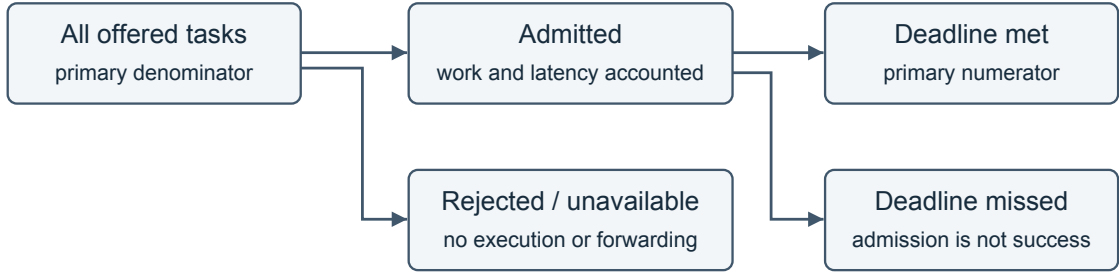
For each offered task i , let $a[i], y[i], z[i, c] \in \{0, 1\}$ indicate admission, recorded deadline success and non-admission category c . Categories are mutually exclusive. An admitted deadline miss has $a[i] = 1, y[i] = 0$ and all $z[i, c] = 0$. The primary outcome and intended accounting contract are:

$$A = \frac{N_{\text{met}}}{N_{\text{offered}}} = \frac{\sum_i y[i]}{N_{\text{offered}}}; \quad \text{reported percentage} = 100A. \quad (1)$$

$$a[i] + \sum_c z[i, c] = 1; \quad y[i] \leq a[i]; \quad N_{\text{offered}} = N_{\text{admitted}} + N_{\text{not admitted}}. \quad (2)$$

Non-admission comprises local queue rejection, V2V queue rejection or unavailability, and V2I unavailability, backlog rejection or capacity rejection. After preliminary eligibility, gate rejection takes precedence when both tests fail. In the three-pass implementation, capacity rejection is the residual label for other excluded candidates; a non-fixed float32 mask can produce this label without a full queue (Appendix C). Rejected tasks reserve no work (Figure 3). Admitted-task attainment and latency summaries concern different populations and must not replace the all-offered outcome.

The inherited gate tests existing effective workload, including the candidate offset $O[i]$, against deadline $D[i]$:



Exactly one admission or nonadmission outcome per offered task; no silent loss.

Figure 3: Offered tasks remain in the primary denominator. Admission, execution assignment and deadline outcome are separate fields. Each offer ends in admission or non-admission.

$$W[r] + O[i] < D[i], \quad \text{together with an available task-count place.} \quad (3)$$

The inequality is strict. It excludes the candidate's service, radio transfer and forwarding cost, and therefore is a backlog filter rather than an end-to-end feasibility guarantee. Candidate ordering and offsets differ between algorithms (§ 2.5). Admission uses positive ingress link quality as its coarse radio condition; the latency model additionally requires quality at least 0.15 for a usable transmission. Consequently, admission can still lead to a transmission-related deadline miss.

For viable V2I execution, the modelled response latency is:

$$L_{\text{raw}}[i] = T(\text{input}[i], q, b) + W[r] + O[i] + s[i, r] + T(0.001, q, b) + f \cdot \mathbf{1}(r \neq \text{ingress}[i]). \quad (4)$$

Here r is the execution RSU, q is ingress-link quality, b is ingress-link capacity in Mbps, $s[i, r]$ is service time, and f is the fixed forwarding cost in ms. $W[r]$ is entry backlog; $O[i]$ adds only earlier admissions in this substep. The 0.001 MB return uses the same link parameters. Appendix F gives transfer constants and the legacy scoring/penalty path; success uses $L[i] \leq D[i]$.

For each queue over an interval, work accounting requires:

$$W_{\text{initial}} + \sum \text{admitted service} = \sum \text{served service} + W_{\text{final}}. \quad (5)$$

After the fifth substep each queue drains up to 1,000 ms of work; Appendix F gives the count-update rule.

2.5 Placement algorithms and their implementation

The inherited identifiers do not define canonical scheduling algorithms: `jsq` selects least service workload rather than shortest task count, and `dla` combines that common-target selector with Equation (3). The evaluator’s own help text and source comment document that `jsq` and `dla` broadcast one `argmin` per substep to every V2I candidate, and that a randomised two-choice mode (`p2c`, `dla_p2c`) exists as a per-vehicle spreading alternative. The gate-off `p2c` mode was not evaluated; `dla_p2c` was added in the declared follow-ups. Table 3 makes their execution destination and gate switch explicit [S2], [S4].

Table 3: Infrastructure conditions. All retain strongest-link radio ingress. Queue eligibility is refreshed at each substep except in historical off mode. Radio links are sampled once per batch; causal policies additionally update reservations after each admission.

Mode	Execution destination	Backlog gate	Coarse eligibility
off	Ingress	Off	Step entry
jsq	Common least-workload target	Off	Substep entry
ingress_dla	Ingress	On	Substep entry
dla	Common least-workload target	On	Substep entry
per_task_dla	Causal least-workload target	On	Substep entry
causal_round_robin	Persistent cyclic target	On	Substep entry
p2c (not evaluated)	Two samples per vehicle, least workload	Off	Substep entry
dla_p2c	Two samples per vehicle, least workload	On	Substep entry

Both algorithms operate once per task substep on entry state W, Q . Candidates follow padded-vehicle order, not arrival-time or deadline priority, with fixed actor action and radio ingress. Every `argmin` considers all RSUs and breaks ties by lowest index; a full selected RSU is not skipped or retried. Two-choice samples RSUs with replacement. “Causal” denotes decisions that see earlier admitted reservations, not causal inference.

Proposition 1 supplies an exact-model guarantee under stated assumptions. For common-target reconciliation, let $E[i]$ mean active V2I attempt, positive ingress quality and $Q[r[i]] < C$ at substep entry. For an admission mask M , define $O[i, M]$ as the sum of $s[j, r[i]]$ over earlier $j < i$ with the same target and $M[j]$ true; $H[i, M]$ is their count. Offsets are exclusive: a candidate’s own service is omitted. Algorithm 1 performs exactly three simultaneous replacements.

Target selection never changes during reconciliation. The retained mask and recomputed offsets determine recorded processing. Section 3.5 distinguishes the exact guarantee from its float32 boundary. Ingress with the gate uses the same offset/reconciliation machinery

Algorithm 1 common-target placement with gate (one substep)

```
r[i] = lowest-index argmin(W), shared by all candidates
M = E
repeat exactly three times:
    compute O[i,M] and H[i,M] from the previous whole mask
    M_new[i] = E[i] and (W[r[i]] + O[i,M] < D[i])
                and (H[i,M] < C - Q[r[i]])
    M = M_new simultaneously
recompute final offsets O[i,M] for latency
label E-candidates failing the final workload test as gate failures
label remaining E-candidates excluded by M as capacity failures
commit each M-admitted task and its full service once
unavailable or rejected tasks reserve nothing
```

with individual ingress targets instead of the common argmin. Gate-off modes omit the workload test.

Algorithm 2 causal per-task placement with gate (one substep)

```
repeat exactly three complete passes:
    working_W = W; working_Q = Q      # restart, not cumulative
    for i in ascending padded-vehicle order:
        r[i] = lowest-index argmin(working_W)
        O[i] = working_W[r[i]] - W[r[i]]
        require active V2I, positive quality and Q[r[i]] < C
        test working_W[r[i]] < D[i], then working_Q[r[i]] < C
        if admitted:
            working_W[r[i]] += s[i,r[i]]
            working_Q[r[i]] += 1
        otherwise, for an offered V2I attempt, record
            unavailability or rejection; reserve nothing
retain the last pass; commit its admitted work once
```

Per-task placement reselects the destination using workloads updated after each admission. One pass is self-consistent; the configured repeated passes restart identically and commit once. The delayed-report variant in Section 2.7 changes selection information while retaining live admission.

Figure 4 is a constructed mechanics example: three empty identical RSUs, four viable 40 ms tasks with 100 ms deadlines, nonbinding capacity and negligible communication. Common-target fixes RSU 0 and admits three tasks with earlier-work offsets 0, 40 and 80 ms. Their completions are 40, 80 and 120 ms: the third passes the backlog gate but misses its deadline, and the fourth is rejected. Work before draining is (120, 0, 0) ms. Per-task

selects RSUs (0, 1, 2, 0), leaves (80, 40, 40) ms and completes all four on time. These simplified services illustrate mechanics, not observed task outcomes.

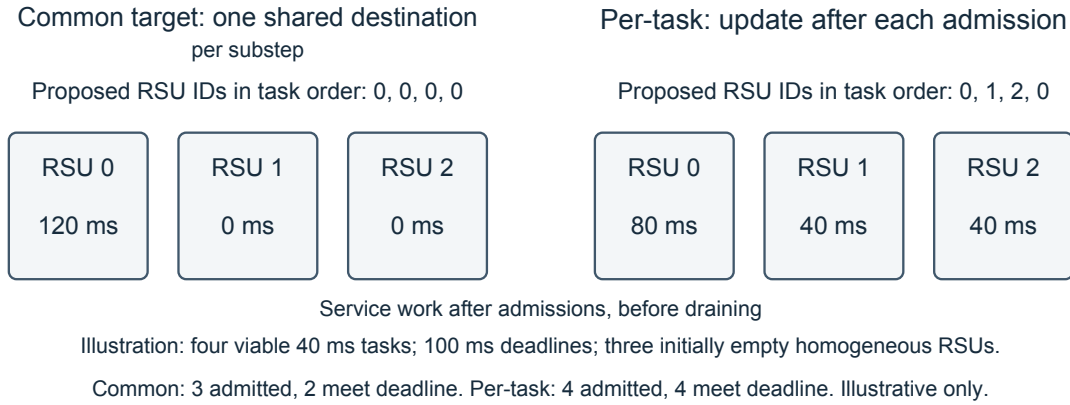


Figure 4: Four illustrative proposals. The common destination remains fixed through reconciliation; causal placement updates after admissions. The accompanying worked example states admissions and deadline outcomes under deliberately simplified assumptions.

2.6 Experimental progression and inference

Table 4: Completed study stages and their replication units.

Code	Descriptive name	Scenario	Replication unit	Section
E0	Accounting validation	Incident	Recorded validation runs	3.1
E1	Capacity scoping	Incident	Matched fleet draw	3.1
E2	Exploratory single-draw comparison	Incident	Inspected fleet draw	3.2
E2b	Ingress-with-gate control	Incident	Same inspected draw	3.2
E2c	Incident four-draw common-target replication	Incident	New fleet draw	3.2
E2d	Incident per-task extension	Incident	Same replication draws	3.2
—	Morning pilot	Morning	Inspected fleet draw	3.3
—	Morning four-draw replication	Morning	New fleet draw	3.3
—	Sealed eight-block confirmation	Morning	Joint fleet/evaluator block	3.4
7–9	Declared follow-ups	Morning	Reused joint block	3.4
—	Three-trace study	Weekend, evening, event	Joint block within trace	3.4

E0 supplies the recorded measurement foundation, while E1 compares three queue limits at fixed service over five matched draws. E2 and E2b explore placement and admission on one draw. The incident replication (E2c) compares common-target with ingress on new fleet seeds 1–4. After inspecting these outcomes, I added per-task placement on the same

draws in the incident extension (E2d), reusing the replication controls. This makes E2d an adaptive extension [S0]–[S3].

The morning pilot uses seed 1 for ingress and per-task placement. After inspecting it, the replication declares seeds 0, 2, 3 and 4 and all three arms, yielding twelve new full runs. The primary contrasts are per-task minus ingress, common-target minus ingress, and per-task minus common-target. The pilot is excluded; a supplementary two-arm five-draw summary is descriptive. The local protocol and manifest preceded the new outcomes, but there was no external preregistration [40] [S7], [S8].

Each fleet draw supplies one paired percentage-point difference d_b , with equal draw weights. The interval is $\bar{d} \pm t_{1-\alpha/(2m), n-1} s_d / \sqrt{n}$, where s_d is the sample standard deviation of the n paired differences (variance divisor $n - 1$). Individual 95% intervals use $m = 1$; the morning three-contrast Bonferroni family uses $m = 3$, $\alpha = 0.05$ and $df=3$ [41]. Reversal requires the simultaneous common-target-minus-ingress interval below zero and per-task-minus-ingress above zero. Four draws cannot strongly assess independence or approximate normality; these are conditional parametric summaries.

The confirmation uses eight seed pairs, (100,200) through (107,207), for fleet and evaluator randomness. Four arms per pair—ingress, common-target, per-task and round-robin—give 32 runs and eight differences per contrast. Controls, rotating order and analysis were sealed before outcomes [40]. The three contrasts are per-task minus ingress, common-target minus ingress and per-task minus round-robin, with equal weights and Bonferroni simultaneous 95% intervals, $df=7$. The same reversal criterion applies; independence and approximately normal paired effects remain assumptions [S16].

2.7 Sensitivities, mechanism audit and validation

The state-information pilot uses incident seed 1 and report ages 0, 100, 500 and 1,000 ms, following Dr Sampaio’s suggested positive ages. For previous post-admission workload B and $0 < d \leq 1,000$ ms, the report is $\max(B - (1,000 - d), 0)$: only $1,000 - d$ ms of service has elapsed by the report time. For example, $B = 600$, $d = 500$ gives 100 ms. This assumes continuous service between batches. Zero age and startup use live state; startup records age zero. Current-batch reservations are added immediately, and admission always uses live queues [S5].

Appendix F records the four forwarding probes and their qualification.

Execution fields record admitted assignment, not departures. Morning checks covered matched inputs/actions and full/probe prefixes, with logit tolerance 10^{-5} and workload-balance error up to about 0.0017 ms [S8], [S13].

A retrospective audit cross-checks admission, categories, latency and assignments; Appendix B records its coverage and the A/B/C diagnostic design.

2.8 Cyclic comparator and retrospective measurement design

Causal round-robin isolates workload awareness from spreading. Its pointer starts at RSU 0, persists across substeps/batches, and advances modulo R for every active positive-radio V2I attempt, including rejections. Unavailable radio and padding do not advance it; there is no retry or full-node skipping. Order, sampled work, gate/capacity, precision, accounting, forwarding and drain match per-task placement. Repeated passes restart identically; the pointer commits once. A versioned evaluator preserves frozen implementations. Qualification checks compatibility, shared inputs, admission accounting and restart; Appendix E bounds coverage [S15], [S16].

Scheduler-only timing follows the protocol in Appendix D over sixteen configurations [42].

Retrospective type aggregation retains all types and draws without subgroup significance tests, using the eight authenticated ingress/per-task files from the original four-draw morning sample. Categories supply admission, gate rejection and admitted misses; type totals must reproduce all-task summaries and paired net changes.

2.9 Method selection and alternatives

Table 5 makes the design trade-offs explicit, using the controls and limitations already described. The alternatives provide a retrospective comparison of methodological choices; they were not all evaluated.

Table 5: Method selection and alternatives. Reasons refer to the implemented study, with consequences limiting interpretation.

Choice	Considered alternative	Reason for selection	Consequence
Frozen actor with matched inputs	Retrain per arm	Isolates infrastructure effect	No claim about co-adapted policies
Offered-task denominator	Admitted-task success	Reject-all scores zero; selective admission cannot hide failures	Rates lower than admitted-only figures
Fleet draws / joint blocks	Task-level tests	Within-run tasks are dependent	Wide intervals; few degrees of freedom
Student- t with Bonferroni	Bootstrap or permutation	Four and eight paired replicates; prespecified three-contrast family	Conservative intervals; parametric assumptions remain

Choice	Considered alternative	Reason for selection	Consequence
Causal round-robin control	Random or two-choice (d1a_p2c) spreading	Isolates workload awareness with identical admission kernel; Section 3.2 explains comparator development	Two-choice evaluated in declared follow-ups (Section 3.4)
Fixed forwarding charge on saved records	Network simulation	Admission, queues and actions unchanged; bounded qualification	No congestion or feedback modelled
Absolute capacity 6,220 per RSU	Ratio to fleet width	Avoids confounding capacity with fleet size	Morning limit non-binding
Trace-driven JAX evaluator	Re-running SUMO per arm	Identical mobility across arms; deterministic replay	No traffic response to offloading

3 Evaluation and Reflection

3.1 Validation and scoping: accounting and historical capacity

This section assesses O1 and establishes the measurement basis for RQ1.

The old evaluator could score eligible V2I tasks while adding only a capacity-limited fraction of their combined service to the queue, without identifying individual refusals. I identified this scoring/admission mismatch; Putra’s 5 August repair introduced per-task admission/rejection and full enqueueing of accepted work. E0 passed its recorded accounting checks. Appendix B illustrates the repair and its scope [S0], [S12].

A later source audit found a remaining exception in historical off scoring, reused by E0, E1 and E2: coarse eligibility reached latency scoring while final admission governed enqueueing. Capacity-rejected tasks occurred, so the affected path was exercised; summaries cannot show whether any were counted as deadline successes. The original task arrays are unavailable. Retained summaries reproduce the historical intervals but cannot quantify this scoring impact; later gate-enabled evidence does not recover those missing observations [S13].

The historical capacity contrast is statistically inconclusive and retains this measurement qualification. Appendix B gives its interval, admitted-latency summaries and the limits of correcting saved history.

3.2 From admission decomposition to the incident reversal

The incident comparisons address RQ1 through O2 and O3.

Setting. E2, the exploratory comparison on seed 0, showed offered attainment of approximately 68.362% for strongest-link execution without the deadline gate, 67.568% for inherited least-busy placement without the gate, and 69.494% for inherited least-busy placement with the gate. The last comparison changed both placement and admission relative to the strongest-link reference. Its improvement could not establish that load balancing itself helped [S2].

E2b added ingress execution with the same deadline gate, reaching approximately 71.577%. The strongest-link contrast was +3.215 points and the common-target contrast +1.926 points, giving an interaction of -1.290 points. However, `off` uses step-entry coarse saturation eligibility while `ingress_dla` refreshes eligibility per substep. Their contrast and interaction bundle the gate with this timing difference and retain the legacy scoring-mask exception (§ 2.4). The manifest records 138 unavailable V2I attempts among 13,076,234 offered tasks in `off`; that count does not bound downstream effects. This control identifies a stronger comparator without assigning its entire gain solely to the gate. Subsequent gate-held placement comparisons use live eligibility and refined scoring masks in all arms. In the exploratory draw, common-target remained 2.083 points below ingress with the gate.

Results. The incident replication (E2c) tested common-target against ingress on four new fleet draws, excluding discovery seed 0. Its mean difference was -2.122 points, with an individual 95% interval of $[-2.234, -2.011]$; all four paired differences were negative.

The evaluator’s documentation records that the inherited scheduler selects one least-workload target per task substep and assigns it to every V2I candidate in that substep. I therefore chose a causal per-task implementation to test reselecting the destination for each task using workloads updated after earlier admissions, while keeping the least-workload criterion fixed. The incident extension (E2d) used the same actor, trace, gate predicate, queue ceiling, service and forwarding controls. Table 6 retains all four draw-level results. The evaluator’s randomised two-choice mode was evaluated later (Section 3.4) [S3].

Table 6: Incident offered-task deadline attainment, expressed as percentages. E2d reused the exact E2c ingress and common-target controls; these are four paired draws, not separate sets of independent controls.

Fleet seed	Ingress	Common-target	Per-task	Per-task – ingress, pp
1	72.003	69.794	72.467	+0.464
2	70.369	68.317	70.956	+0.587
3	70.298	68.153	70.805	+0.507
4	70.887	68.805	71.438	+0.551

Per-task exceeds ingress by 0.527 points, individual 95% interval [+0.442, +0.612], and common-target by 2.649 points, interval [+2.621, +2.678]. RQ1 therefore yields opposite rankings for the two implementations. These intervals summarise an adaptive extension on examined draws; the morning protocol supplies prospective replication.

The two contrasts have different interpretations. The larger +2.649-point advantage over common-target diagnoses the inherited implementation. Against ingress attainment of 70.889%, the +0.527-point difference corresponds to about 53 extra successes per 10,000 offered tasks, or a 0.744% relative increase. These conversions describe the observed mean; neither arm establishes optimal dispatch.

3.3 Morning replication with the inspected pilot excluded

The morning replication addresses RQ2 and O3 with prospectively declared comparisons.

Setting. The inspected morning pilot offered 1,744,116 tasks and produced 91.075% ingress versus 94.028% per-task attainment: +2.953 points or 51,503 successes. It motivated replication [S7].

Results. The morning protocol declared four new seeds and all three arms before outcomes. Table 7 reproduces common-target < ingress < per-task in every draw. Mean attainment is 85.655%, 88.887% and 92.785%, respectively. The ingress and per-task means are lower than in the pilot, illustrating why the inspected draw was excluded from primary inference [S8].

Table 7: Morning primary replication. Seed 1 is absent because it was the inspected pilot; each row is a matched fleet draw on the same morning trace.

Fleet seed	Ingress, %	Common-target, %	Per-task, %	Per-task – ingress, pp
0	88.817	85.660	92.705	+3.888
2	87.536	83.537	92.060	+4.524
3	89.082	85.862	92.978	+3.896
4	90.113	87.559	93.399	+3.286

Table 8’s simultaneous per-task-minus-ingress interval is positive and common-target-minus-ingress interval negative, satisfying the prespecified reversal criterion. Figure 5 displays these primary results.

Table 8: Morning paired contrasts in percentage points. Individual intervals are two-sided 95% Student-*t* intervals; simultaneous intervals use the prespecified three-contrast Bonferroni adjustment. Replication uses four fleet draws. The simultaneous intervals determine the declared three-contrast decision.

Contrast	Mean	Individual 95% interval	Simultaneous 95% interval
Per-task – ingress	+3.899	[+3.094, +4.703]	[+2.671, +5.126]
Common-target – ingress	−3.232	[−4.176, −2.289]	[−4.672, −1.793]
Per-task – common-target	+7.131	[+5.385, +8.877]	[+4.466, +9.795]

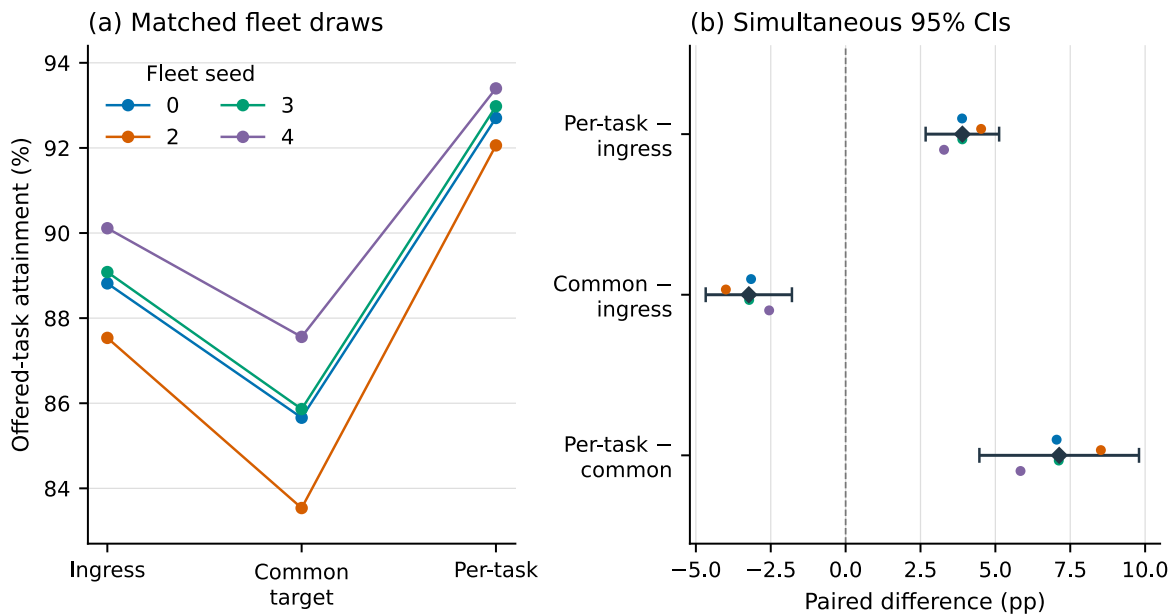


Figure 5: Redrawn from archived morning primary data. Left: within-draw attainment. Right: draw differences, mean diamonds and three-contrast Bonferroni intervals. The inspected pilot is excluded; uncertainty concerns four newly declared fleet draws, conditional on the frozen actor and morning scenario [S8].

The five-draw ingress/per-task mean including the pilot is +3.709 points descriptively, outside Table 8; no common-target pilot result is available.

The larger morning effect cannot be attributed solely to density: RSU layout/count, date, duration, padded fleet width and entry conventions also change. Separate scenario analyses support recurrence. The morning gain is about 390 extra successes per 10,000 offered, a 4.386% relative increase over ingress.

3.4 Confirmation under joint randomness and comparison with cyclic spreading

The confirmation extends RQ2 and O2–O3 to joint fleet/evaluator randomness and adds a cyclic spreading comparator.

Setting. Evaluator seed varies observation descriptors, arrivals, operational types/sizes, fading and service wobble; fleet seed varies tier, EV status and initial SoC. The estimand is the total scheduling intervention under frozen weights, without forcing replay or guaranteeing future action equality. Small observation and logit differences occurred in block 1. All four arms shared the required exogenous arrays exactly within each block, and actions matched exactly in all 24 non-reference arm/block comparisons.

Results. All 32 full morning cells and eight block-control receipts passed after bounded qualification, with no failed attempts, retries or exclusions. Table 9 reports the declared contrasts; Appendix E retains every block [S16].

Table 9: New morning confirmation, eight paired joint-seed blocks with equal weighting. Two-sided Bonferroni simultaneous 95% Student-*t* intervals cover the three predeclared contrasts; positive differences favour the first arm. Both reversal directions and the cyclic-control increment belong to this family.

Contrast	Mean difference, pp	Simultaneous 95% interval, pp
Per-task – ingress	+4.137	[+3.556, +4.717]
Common-target – ingress	−3.504	[−3.980, −3.028]
Per-task – round-robin	+0.631	[+0.511, +0.750]

Both directional requirements satisfy the declared reversal criterion within the same simultaneous family. Relative to ingress, the mean per-task difference corresponds to +413.7 successes per 10,000 offered tasks. On the morning trace, this mean effect corresponds to about 24,000 additional tasks per hour meeting their deadline (Table E1). Per-task placement also outperformed cyclic spreading by 63.1 successes per 10,000 offered tasks, although its scheduler-only cost was higher on the separate fixtures in Table D2.

Figure 6 shows all four arms in each of the eight blocks. Equal block weighting gives mean attainment of 88.372% for ingress, 91.879% for round-robin and 92.509% for per-task placement. The descriptive round-robin-minus-ingress difference is +3.506 percentage points, and the further per-task-minus-round-robin increment is +0.631 points. Their sum is the +4.137-point per-task-minus-ingress difference [S16].

Using unrounded means, **84.8% of the ingress-to-per-task difference lies between ingress and round-robin, and 15.2% between round-robin and per-task.** These descriptive shares of arm-mean differences neither identify causal mechanism shares nor constitute a fourth declared inferential contrast. The per-task versus causal round-robin comparison provides the closer assessment of workload awareness: cyclic spreading captures most of the observed ingress gain, and workload-aware selection adds a smaller, consistently positive increment.

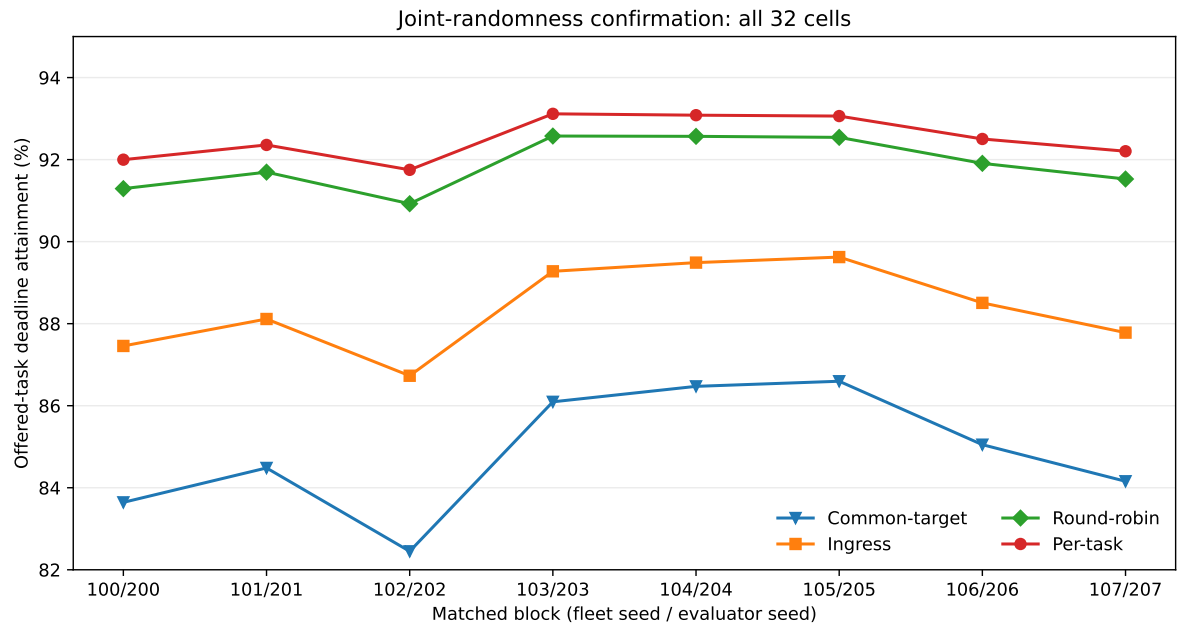


Figure 6: Offered-task deadline attainment recomputed as 100 times successes divided by offers for each of the 32 archived cells. Lines connect the same policy across eight matched blocks; block labels show fleet/evaluator seeds. No tasks are treated as independent replications and no uncertainty intervals are inferred from the line segments. Every block retains common-target below ingress and per-task above round-robin; Table 9 supplies the declared simultaneous intervals [S16].

The simultaneous intervals use a critical value of 3.13 with seven degrees of freedom (Appendix E). The confirmation extends the original fleet-conditional finding to new evaluator streams.

Declared follow-ups. Three studies reused the confirmation’s eight joint blocks under a protocol sealed before new outcomes. These additions are outside the original family; all ten new contrasts share a Bonferroni adjustment (Table 10).

Study 7 added the evaluator’s d1a_p2c mode. Per-task exceeded it by +0.649 points; two-choice did not differ conclusively from round-robin. Study 8 halved RSU service rate: the per-task advantage over round-robin increased by +0.193 points, supporting the tested lower-rate effect but not a service-rate curve. Study 9 used a second saved Model-C checkpoint trained on the UK-fleet distribution, sharing training seed 100. It retained common-target < ingress < round-robin < per-task, without establishing robustness to independent training. The half-speed and second-checkpoint conditions were separate [S18].

Table 10: Follow-up contrasts on the same eight blocks: post-hoc additions with a ten-contrast Bonferroni family. Half-speed and second-actor studies are separate conditions and are not combined. Comparisons are paired within each condition; the original confirmation retains its separate family.

Study	Contrast	Mean difference (pp)	All-ten simultaneous 95% interval
7	Per-task — two-choice	+0.649	[+0.517, +0.781]
7	Two-choice — ingress	+3.488	[+2.869, +4.107]
7	Two-choice — round-robin	−0.018	[−0.048, +0.012]
8	Per-task — round-robin at half speed	+0.824	[+0.787, +0.860]
8	Change in per-task — round-robin gap	+0.193	[+0.034, +0.352]
9	Per-task — ingress	+3.539	[+2.861, +4.217]
9	Per-task — round-robin	+0.550	[+0.432, +0.668]

Three-trace study. Five arms were tested on eight blocks for each of three unused traces: weekend (15 September 2024, 12:00–21:00), evening peak (15 October 2024, 14:00–21:00) and event night (18 September 2024, 17:30–24:00). All 120 cells completed without failures under a protocol sealed before outcomes. Table 11 gives the fifteen-contrast-family intervals. The common-target < ingress < round-robin < per-task ranking recurred; two-choice never exceeded round-robin in trace means.

In order of increasing average active-vehicle count—event night, weekend, evening peak, morning—the per-task advantage over round-robin was 0.43, 0.51, 0.61 and 0.63 percentage points. This is descriptive: event night has twelve RSUs against nine or ten, and only weekend among the new traces carries the entry-channel fix. Evening peak and event night retain the incident trace’s legacy slot-reuse convention. These bundled differences prevent attributing the pattern to density alone [S19].

Table 11: Three further Manchester traces, each with eight paired blocks and five arms. All-fifteen simultaneous 95% Student- t intervals use one Bonferroni family across the fifteen new contrasts; the complete analysis also reports two-choice minus round-robin. Each trace is assessed separately; density ordering is descriptive, with no pooling or cross-trace test [S19].

Trace	Contrast	Mean difference (pp)	All-fifteen simultaneous 95% interval
Weekend	Per-task – ingress	+3.728	[+2.313, +5.144]
Weekend	Common-target – ingress	−2.996	[−4.230, −1.763]
Weekend	Per-task – round-robin	+0.513	[+0.284, +0.742]
Weekend	Per-task – two-choice	+0.580	[+0.377, +0.784]
PM peak	Per-task – ingress	+4.359	[+2.406, +6.311]
PM peak	Common-target – ingress	−3.594	[−5.403, −1.784]
PM peak	Per-task – round-robin	+0.606	[+0.160, +1.051]
PM peak	Per-task – two-choice	+0.646	[+0.290, +1.003]
Event night	Per-task – ingress	+3.735	[+2.385, +5.085]
Event night	Common-target – ingress	−3.593	[−4.916, −2.269]
Event night	Per-task – round-robin	+0.428	[+0.255, +0.600]
Event night	Per-task – two-choice	+0.534	[+0.350, +0.717]

3.5 Why destination semantics matter

The destination audit and exact-model analysis address RQ1 and O4. A fixed point is an admission mask unchanged by another update. Proposition 1 explains why three simultaneous updates suffice for two deadline values when destinations stay fixed; it does not equate common-target placement with per-task reselection.

Common-target placement assigned morning work only to RSUs 0–4, whereas causal per-task placement used all nine. With K substeps and R RSUs, selecting one destination per substep admits new work to at most $\min(K, R)$ destinations in a batch. Empty starts, lowest-index ties, positive admissions and no intervening drain further produce the recurring low-index prefix. Under the exact gate/service model, workload remains below 538.889 ms before the 1,000 ms drain, explaining the recurring empty starts. Appendix C retains the reconstructed states and Figure 9. The live-state mechanism differs from stale-information herding [22]; federation benefits are implementation-specific [25] [S8], [S9].

Proposition 1 (fixed-target admission in the studied exact model). *Fix candidate order, eligibility, destinations, initial nonnegative workload/count and nonnegative service demands, with strict backlog and exclusive count-capacity tests and no intervening drain. At each nonempty destination, let n be the eligible candidate count and d the number of distinct deadlines. Starting from the eligibility mask, simultaneous replacement reaches the unique causal fixed point within $\min(n, 2d - 1)$ replacements; destinations decouple.*

The unique causal fixed point is the sequential accept/reject result at those same fixed destinations. The two production deadlines, 100 and 500 ms, give at most three replacements.

This exact-arithmetic result guarantees neither optimal placement nor timely completion; Appendix C proves the bound and states the float32 boundary [43].

Matched task accounting in the original morning replication (seeds 0, 2, 3 and 4) records 284,821 gained successes and 12,842 losses, giving 271,979 net successes. Most gains were tasks rejected by the ingress backlog gate. Tables C2 and C3 retain both gains and losses. Diverging later queues make these descriptive transitions, not causal effects of individual decisions.

3.6 Bounded state-information and forwarding sensitivities

The completed sensitivities address RQ3 and O4 by varying reported state and fixed forwarding cost within their stated models.

Report-age results. In incident seed 1, fresh per-task attainment was 72.47%, versus ingress 72.00%. The 100 ms report left attainment unchanged at 72.47% (+0.000 points). The 500 ms and 1,000 ms reports gave 72.47% and 72.46%, changes of +0.002 and −0.012 points from fresh [S5].

Treatment activation. Every 100 ms report equalled fresh state over 35,990 non-startup RSU/batch observations: queues had emptied, making that treatment inactive. Reports differed at 500 ms in 79.58% of observations and at 1,000 ms in all.

Fixed forwarding overheads 0, 1, 2.5, 5 and 10 ms yielded 72.47%, 72.45%, 72.42%, 72.37% and 72.29% attainment. Changes from the fresh zero-overhead value were +0.000, −0.019, −0.048, −0.095 and −0.181 points, respectively. At 10 ms the advantage was 0.283 points, or 36,942 successes over ingress, despite 23,689 extra misses relative to zero overhead [S6].

These are saved-record transformations qualified by four short direct probes. Forwarding charges do not feed back into admission, queues or actions in that path; the calculation models neither congestion nor postponed arrivals. RQ3 supports the tested information and fixed-cost models, not an extrapolated break-even threshold or network design.

3.7 Computational trade-offs and research implementation

This section assesses O5 and the computational cost associated with the attainment comparisons in RQ2.

Across scheduler-only fixtures, per-task median execution times were 0.05–0.62 ms per five-substep batch, compared with 1–6 ms for the three-replacement arms (Table D2). The scan traverses all padded positions; round-robin cost less. Appendix D explains the

compiler-dependent ordering. These host timings exclude compilation and are distinct from simulated task latency.

Compilation took 58–267 ms per scheduler configuration, measured separately from lowering, warm-up and execution. The fixtures do not establish full-evaluator costs, peak memory or roadside timing guarantees [S15].

The research path combines a trace-driven JAX evaluator, versioned schedulers, sealed runner and separate validation/analysis [44]. The TrafficTwin product implements scenario configuration, validation and export, what-if preparation, comparison and provenance through Streamlit, typed services and CLI workflows [S11], [S17].

Contracts require admission before recorded execution and a passing validation receipt before accepting a scientific cell. The runner binds source, seeds and outputs.

Table A2 records component scope and required conditions; line counts do not measure individually authored code or establish reliability.

Version control, documented software checks and preserved analysis records support inspectability and reproducibility [45], [46]. No Streamlit exceptions or horizontal overflow were observed in an automated inspection of five TrafficTwin pages at a 1440×1000 desktop viewport in light theme (inspection record). This inspection did not assess usability. Figures 7 and 8 demonstrate imported-data validation, comparison and provenance; numerical claims rest on the bound campaign outputs.

My design and implementation contributions are listed in Section 1.3. The author-check record documents a block-0 recomputation from offered counts, interval checks for Tables 8 and 9, worked examples for Proposition 1 and Table C1, and a trace of common-target selection in the frozen evaluator [S12].

Table 12 records the limits of aggregate departures and global work with immediate reservations. Separate lifecycle fields distinguish placement, admission and success.

(a) TOS Data Import

TOS Data package directory

/Users/akashx/Downloads/diss_mat/tos-data-full

Inspect Package

Deep Validate NPZ Contracts

△ ACCEPTED WITH WARNINGS

IMPORTED SIMULATION

Evaluation runs

300

Per-step files

60

Per-task showcases

6

Trace files

5

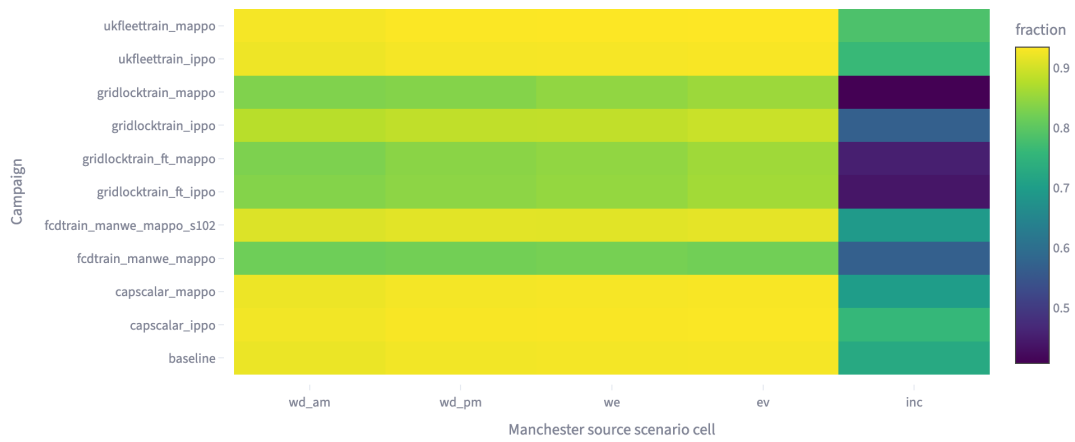
Training curves

66

Engine: v2_post_nrsus_fix | Package commit: a75bbdb1a956... | Fingerprint: 6424ce5375b4...

(b) External deadline-success matrix and comparison

Evaluation matrix



> Measure definition and limitations

Paired campaign comparison

Baseline campaign

baseline

Variation campaign

capsalar_ippo

cell	paired n	baseline mean	variation mean	mean delta	sample SD of deltas	compatibility
wd_am	5	0.9192	0.9246	0.0054	0.0007	obs_variant differs: onehot17 vs capsalar13
wd_pm	5	0.9248	0.9295	0.0047	0.0002	obs_variant differs: onehot17 vs capsalar13
we	5	0.9261	0.9304	0.0043	0.0002	obs_variant differs: onehot17 vs capsalar13
ev	5	0.9275	0.9315	0.004	0.0002	obs_variant differs: onehot17 vs capsalar13
inc	5	0.7247	0.7587	0.034	0.0077	obs_variant differs: onehot17 vs capsalar13

Figure 7: TrafficTwin with real data. (a) An external evaluation package (R. P. Putra, Paper 2B data, tos-data commit a75bbdb, 18–23 July 2026; engine v2_post_nrsus_fix) validated read-only, with its provenance line. (b) That package’s 300 evaluation runs across five Manchester traces as a deadline-success matrix with the paired campaign comparison beneath. These are external numbers from an engine since revised (vec_env 9db5096, 3 Aug 2026; df022e6/82d3ecf, 5 Aug 2026), shown to demonstrate provenance and comparison workflows, not as results of this dissertation. Panel (b)’s compatibility column records a declared observation-encoding difference (onehot17 versus capsalar13) flagged by the adapter, not an error. Panels (a)–(b) contain imported simulation evidence; Figure 8 shows the sealed dissertation confirmation. These are original interface captures.

(c) Sealed eight-block dissertation contrasts

Experimental Results

Workflow demonstration · completed simulation evidence · 8 September 2026

> Import study evidence

Joint-randomness confirmation

Morning trace · frozen vehicle actor · four infrastructure scheduling policies · fleet and evaluator seeds vary together across eight paired blocks

Paired blocks	Completed runs	Policies	Cell receipts matched
8	32	4	32

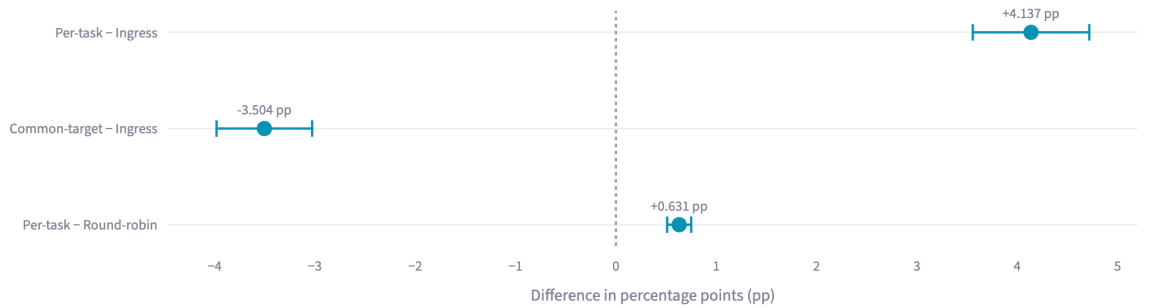
Original study files match their recorded hashes. Compact result arithmetic and receipt bindings checked.

Task-level validation is recorded in the historical receipts; raw simulation arrays are not rechecked by this view.

Paired comparisonsRuns & blocksEvidence & downloads

Change in deadline attainment

Successes ÷ all offered tasks. Each paired block has equal weight; individual tasks are not statistical replicates.



Comparison	Mean difference (pp)	Simultaneous 95% interval (pp)	Paired blocks
Per-task - Ingress	+4.137	[+3.556, +4.717]	8
Common-target - Ingress	-3.504	[-3.980, -3.028]	8
Per-task - Round-robin	+0.631	[+0.511, +0.750]	8

Original three-contrast family: Bonferroni simultaneous 95% t intervals, 7 degrees of freedom; approximate normality of block differences assumed. This study is separate from prior experiments; results are not pooled.

Figure 8: TrafficTwin with real data. (c) This dissertation’s sealed eight-block contrasts read from the bundled results packet. The platform demonstrates validation, comparison and provenance on genuine inputs; the numerical claims of this report rest on the bound campaign outputs, not on the interface.

Table 12: Validity limits and their consequences for interpretation.

Limitation	Effect on conclusions	Further evidence needed
Batched arrivals, aggregate departures, backlog-only gate	Outcomes describe evaluator deadlines, not physical feasibility	Validate event/transfer timing against measurements
Global actual service work; immediate reservations	Distributed deployability is untested	Measure estimation error and coordination delay
Original four fleet draws; new eight joint blocks; five scenarios of one demand-scaled Manchester network; two checkpoints sharing a training seed; follow-ups reuse the seed pairs	Generalisation is limited to sampled conditions	Test independent networks/training seeds and combined conditions
Bundled scenario changes	Differences cannot be attributed to density alone	Factor-controlled comparisons
Legacy off scoring masks differ	Historical scoring impact and gate-only attribution remain unknown	Original task-level records needed; later runs cannot recover them
Exact arithmetic versus float32; finite fixtures and unobserved intermediate masks	Constructed discrepancies do not establish fleet frequency or causal shares	Recorded intermediate masks and representative numerical diagnostics
Single-draw sensitivities	Robustness is limited to the tested models	Additional paired draws and communication/feedback models
Separately retained raw inputs	Compact checks reproduce summaries, not full task-level execution	Raw inputs and recorded dependencies for full replay (Appendix G)

Appendix G describes compact recomputation and the inputs required for full reproduction [S13], [S16].

4 Conclusion

Infrastructure scheduling implementation reversed the deadline-attainment comparison under fixed actor weights. Across 274 runs, the project compared placement and admission rules using all offered tasks as the outcome population. The sealed confirmation found a +4.137-point per-task advantage over ingress and +0.631 over round-robin (Table 9). These results adapt established scheduling ideas: their contribution is an explicit, tested account of destination selection and reservation timing within one evaluator and network. Against the original objectives, the project clarified offered/admitted accounting and tested scheduling alternatives under matched controls. TrafficTwin implements scenario configuration, validation/export and comparison/evidence workflows. Historical scoring impact remains unresolved, generalisation is limited to the tested model and network, and usability and operator benefit remain unevaluated.

4.1 Analysis of the project approach

Scope and execution. The initial programme connected incident draws, morning replication and joint-seed confirmation. All 32 confirmation cells were valid without retries, with actions matching in 24 non-reference comparisons. The maximum reconstructed RSU service discrepancy was about 0.00012 ms; Appendix E gives the separate vehicle-queue bound.

Methodological challenges. Recovering the inherited evaluator’s semantics required source inspection. The adaptive incident extension motivated separate prospective comparisons. Missing early task arrays still prevent quantifying the historical scoring discrepancy; later retained records support their own campaigns.

4.2 Research answers and interpretation

RQ1: implementation. Common-target and per-task placement produced opposite rankings relative to gate-enabled ingress. The least-workload label alone was insufficient: selecting one shared destination differs from selecting again after each admission. The exploratory gate comparison also bundled eligibility timing and historical scoring differences, so it cannot isolate a gate-only effect.

RQ2: recurrence and workload awareness. Separate morning and joint-seed comparisons reproduced the reversal. Round-robin recovered most of the ingress-to-per-task gain; per-task added a smaller positive increment in every confirmation block and each new trace. Follow-ups extended this result to two-choice placement, half service speed and a second checkpoint. Reused blocks, a shared training seed and bundled scenario differences limit generalisation; arm differences do not identify causal mechanism shares.

RQ3: information and forwarding. Older reports produced small changes with live admission and immediate reservations; 100 ms reports equalled fresh state, so that treatment did not activate. The single-draw fixed-charge calculation retained the per-task advantage through 10 ms. These findings concern the specified report/overhead models, which omit congestion and forwarding feedback.

The exact-model proof and retrospective task accounting answer different questions. The former bounds fixed-destination admission updates; the latter records recovered rejections, admitted misses and smaller losses. Neither establishes a unique causal decomposition of the experimental gain.

4.3 Prioritised future work

First, an independently trained checkpoint, a second traffic network and a service-rate curve beyond the single half-speed point would test whether the workload-aware increment scales as the mechanism predicts.

Second, an event-level service and communication model should be checked against measured timing, including transfers and individual departures. Simulation verification and VEC communication/resource models provide the relevant foundation [9], [26], [36]. The decisive test would be whether the reversal survives a calibrated timing model.

Third, I would separate imperfect workload estimates, acknowledgement delay and admission policy, following imperfect-information routing and digital-twin estimation models [21], [35]. This tests how live admission protects placement based on older reports.

Fourth, I would test independently selected traces and actors under prospective controls and seed-level reporting [30], [31], [47]. Action masking and V2V-helper selection need separate studies [37], [48]; product user evaluation requires its own protocol.

4.4 Reflection on the process

The per-task extension reselected destinations using workloads updated after earlier admissions. It reused incident controls after their outcomes had been inspected, so it was adaptive rather than independent confirmation. Primary inference subsequently used the declared morning replication, excluding the inspected pilot, and separate sealed joint-seed blocks.

Substantial what-if functionality was implemented, including scenario configuration, validation/export and comparison/evidence workflows. A user evaluation was not completed, leaving usability and operator benefit unmeasured.

References

- [1] L. Liu, C. Chen, Q. Pei, S. Maharjan and Y. Zhang. “Vehicular edge computing and networking: A survey.” *Mobile Networks and Applications*, 26(3), 1145–1168, 2021. DOI: 10.1007/s11036-020-01624-1.
- [2] S. Raza, S. Wang, M. Ahmed and M. R. Anwar. “A survey on vehicular edge computing: Architecture, applications, technical issues, and future directions.” *Wireless Communications and Mobile Computing*, 2019, 1–19, 2019. DOI: 10.1155/2019/3159762.
- [3] Y. Mao, C. You, J. Zhang, K. Huang and K. B. Letaief. “A survey on mobile edge computing: The communication perspective.” *IEEE Communications Surveys & Tutorials*, 19(4), 2322–2358, 2017. DOI: 10.1109/COMST.2017.2745201.

- [4] World Health Organization. *Global status report on road safety 2023*. Geneva: WHO, 2023. ISBN 978-92-4-008651-7. Executive summary p. viii and p. 4. Accessed 15 September 2026.
- [5] 3GPP. *Service requirements for enhanced V2X scenarios*. TS 22.186, version 16.2.0, Release 16; ETSI TS 122 186 V16.2.0, November 2020. Clauses 5.2–5.3, Tables 5.2-1 (R.5.2-008) and 5.3-1 (R.5.3-004/005), pp. 9–10. Accessed 15 September 2026.
- [6] C. L. Liu and J. W. Layland. “Scheduling algorithms for multiprogramming in a hard-real-time environment.” *Journal of the ACM*, 20(1), 46–61, 1973. DOI: 10.1145/321738.321743.
- [7] A. Molisch, F. Tufvesson, J. Karedal and C. Mecklenbrauker. “A survey on vehicle-to-vehicle propagation channels.” *IEEE Wireless Communications*, 16(6), 12–22, 2009. DOI: 10.1109/MWC.2009.5361174.
- [8] W. Fan, Y. Zhang, G. Zhou and Y. Liu. “Deep reinforcement learning-based task offloading for vehicular edge computing with flexible RSU-RSU cooperation.” *IEEE Transactions on Intelligent Transportation Systems*, 25(7), 7712–7725, 2024. DOI: 10.1109/TITS.2024.3349546.
- [9] Q. Wu, W. Wang, P. Fan, Q. Fan, J. Wang and K. B. Letaief. “URLLC-
aware resource allocation for heterogeneous vehicular edge computing.” *IEEE Transactions on Vehicular Technology*, 73(8), 11789–11805, 2024. DOI: 10.1109/TVT.2024.3370196. Author version arXiv:2311.18352.
- [10] R. P. Putra, L. Figueiredo Silva, S. Sampaio and P. Sampaio. “Evaluating the value of predictive information in partially observed networked systems: A traffic control case study.” In *Proceedings of the 9th International Conference on Information Science and Systems (ICISS 2026)*, Cambridge, UK, 27–29 August 2026. ACM, 10 pages. ISBN 979-8-4007-2432-9. DOI pending in the camera-ready copy supplied by the first author.
- [11] W. Fan, Y. Yu, C. Bao and Y. Liu. “Vehicular edge intelligence: DRL-based resource orchestration for task inference in vehicle-RSU-edge collaborative networks.” *IEEE Transactions on Mobile Computing*, 24(10), 10927–10944, 2025. DOI: 10.1109/TMC.2025.3572296.
- [12] J. Zhang and K. B. Letaief. “Mobile edge intelligence and computing for the Internet of Vehicles.” *Proceedings of the IEEE*, 108(2), 246–261, 2020. DOI: 10.1109/JPROC.2019.2947490.
- [13] Y. Liu, H. Yu, S. Xie and Y. Zhang. “Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks.” *IEEE Transactions on Vehicular Technology*, 68(11), 11158–11168, 2019. DOI: 10.1109/TVT.2019.2935450.
- [14] Z. Ning, P. Dong, X. Wang, J. J. P. C. Rodrigues and F. Xia. “Deep reinforcement learning for vehicular edge computing: An intelligent offloading system.” *ACM Transactions on Intelligent Systems and Technology*, 10(6), 1–24, 2019. DOI: 10.1145/3317572.
- [15] R. R. Weber. “On the optimal assignment of customers to parallel servers.” *Journal of Applied Probability*, 15(2), 406–413, 1978. DOI: 10.2307/3213411.
- [16] M. Harchol-Balter. *Performance modeling and design of computer systems: Queueing theory in action*. Cambridge University Press, 2013. DOI: 10.1017/CBO9781139226424.
- [17] M. Harchol-Balter, M. E. Crovella and C. D. Murta. “On choosing a task assignment policy for a distributed server system.” *Computer Performance Evaluation (TOOLS 1998)*, LNCS 1469, 231–242, 1998. DOI: 10.1007/3-540-68061-6_19.

- [18] K. Ousterhout, P. Wendell, M. Zaharia and I. Stoica. “Sparrow: Distributed, low latency scheduling.” *24th ACM Symposium on Operating Systems Principles*, 69–84, 2013. DOI: 10.1145/2517349.2522716.
- [19] L. Ying, R. Srikant and X. Kang. “The power of slightly more than one sample in randomized load balancing.” *IEEE INFOCOM*, 1131–1139, 2015. DOI: 10.1109/INFOCOM.2015.7218487.
- [20] M. Mitzenmacher. “The power of two choices in randomized load balancing.” *IEEE Transactions on Parallel and Distributed Systems*, 12(10), 1094–1104, 2001. DOI: 10.1109/71.963420.
- [21] S. Vargaftik, I. Keslassy and A. Orda. “LSQ: Load balancing in large-scale heterogeneous systems with multiple dispatchers.” *IEEE/ACM Transactions on Networking*, 28(3), 1186–1198, 2020. DOI: 10.1109/TNET.2020.2980061.
- [22] M. Mitzenmacher. “How useful is old information?” *IEEE Transactions on Parallel and Distributed Systems*, 11(1), 6–20, 2000. DOI: 10.1109/71.824633.
- [23] M. van der Boor, S. C. Borst, J. S. H. van Leeuwen and D. Mukherjee. “Scalable load balancing in networked systems: A survey of recent advances.” *SIAM Review*, 64(3), 554–622, 2022. DOI: 10.1137/20M1323746.
- [24] J. Niño-Mora. “Admission and routing of soft real-time jobs to multiclusters: Design and comparison of index policies.” *Computers & Operations Research*, 39, 3431–3444, 2012. DOI: 10.1016/j.cor.2012.05.004.
- [25] A. Kovalenko, R. F. Hussain, O. Semiari and M. A. Salehi. “Robust resource allocation using edge computing for vehicle to infrastructure (V2I) networks.” *2019 IEEE 3rd International Conference on Fog and Edge Computing (ICFEC)*, 1–6, 2019. DOI: 10.1109/CFEC.2019.8733151.
- [26] R. G. Sargent. “Verification and validation of simulation models.” *Proceedings of the 2011 Winter Simulation Conference*, 183–198, 2011.
- [27] P. Alvarez Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner and E. Wießner. “Microscopic traffic simulation using SUMO.” *21st International Conference on Intelligent Transportation Systems (ITSC)*, 2575–2582, 2018. DOI: 10.1109/ITSC.2018.8569938.
- [28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov. “Proximal policy optimization algorithms.” arXiv:1707.06347, 2017.
- [29] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen and Y. Wu. “The surprising effectiveness of PPO in cooperative multi-agent games.” *Advances in Neural Information Processing Systems*, 35, 24611–24624, 2022.
- [30] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup and D. Meger. “Deep reinforcement learning that matters.” *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018. DOI: 10.1609/aaai.v32i1.11694.
- [31] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville and M. G. Bellemare. “Deep reinforcement learning at the edge of the statistical precipice.” *Advances in Neural Information Processing Systems*, 34, 29304–29320, 2021.
- [32] R. Gorsane, O. Mahjoub, R. de Kock, R. Dubb, S. Singh and A. Pretorius. “Towards a standardised performance evaluation protocol for cooperative MARL.” *Advances in Neural Information Processing Systems*, 35, 5510–5521, 2022.
- [33] F. Tao, H. Zhang, A. Liu and A. Y. C. Nee. “Digital twin in industry: State-of-the-art.” *IEEE Transactions on Industrial Informatics*, 15(4), 2405–2415, 2019. DOI: 10.1109/TII.2018.2873186.

- [34] S. Dasgupta, M. Rahman, A. D. Lidbe, W. Lu and S. Jones. “A transportation digital-twin approach for adaptive traffic control systems.” arXiv:2109.10863, 2021.
- [35] Y. Xie, Q. Wu and P. Fan. “Digital twin vehicular edge computing network: Task offloading and resource allocation.” *2024 7th International Conference on Information Communication and Signal Processing (ICICSP)*, 1137–1141, 2024. DOI: 10.1109/ICICSP62589.2024.10809180.
- [36] S. Cho, S. I. Choi, S. H. Oh, I. P. Roberts and S. H. Lee. “Autonomous task offloading of vehicular edge computing with parallel computation queues.” *IEEE Transactions on Mobile Computing*, 25(5), 7166–7181, 2026. DOI: 10.1109/TMC.2025.3640244. Author version arXiv:2509.03935v2.
- [37] E. Krijestorac, A. Memedi, T. Higuchi, S. Ucar, O. Altintas and D. Cabric. “Hybrid vehicular and cloud distributed computing: A case for cooperative perception.” *GLOBECOM 2020 – 2020 IEEE Global Communications Conference*, 1–6, 2020. DOI: 10.1109/GLOBECOM42002.2020.9322247.
- [38] S. Sampaio (proposer). “Dynamic resource management for intelligent transportation system applications.” Taught Project Explorer, project 237, The University of Manchester. Accessed 27 May 2026.
- [39] The University of Manchester. *Ethics Decision Tool*. University guidance. Accessed 15 September 2026; checked by the author; outcome: ethics approval not required.
- [40] B. A. Nosek, C. R. Ebersole, A. C. DeHaven and D. T. Mellor. “The preregistration revolution.” *Proceedings of the National Academy of Sciences*, 115(11), 2600–2606, 2018. DOI: 10.1073/pnas.1708274114.
- [41] O. J. Dunn. “Multiple comparisons among means.” *Journal of the American Statistical Association*, 56(293), 52–64, 1961. DOI: 10.1080/01621459.1961.10482090.
- [42] JAX authors. “Benchmarking JAX code.” Official JAX documentation. Accessed 8 September 2026. Used for timing procedure, not scheduler efficacy.
- [43] D. Goldberg. “What every computer scientist should know about floating-point arithmetic.” *ACM Computing Surveys*, 23(1), 5–48, 1991. DOI: 10.1145/103162.103163.
- [44] J. Bradbury et al. “JAX: Composable transformations of Python+NumPy programs.” Software, version 0.4.30, 2024 (first released 2018).
- [45] G. Wilson et al. “Best practices for scientific computing.” *PLOS Biology*, 12(1), e1001745, 2014. DOI: 10.1371/journal.pbio.1001745.
- [46] G. K. Sandve, A. Nekrutenko, J. Taylor and E. Hovig. “Ten simple rules for reproducible computational research.” *PLOS Computational Biology*, 9(10), e1003285, 2013. DOI: 10.1371/journal.pcbi.1003285.
- [47] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché-Buc, E. Fox and H. Larochelle. “Improving reproducibility in machine learning research (a report from the NeurIPS 2019 Reproducibility Program).” *Journal of Machine Learning Research*, 22(164), 1–20, 2021.
- [48] S. Huang and S. Ontañón. “A closer look at invalid action masking in policy gradient algorithms.” *The International FLAIRS Conference Proceedings*, 35, 2022. DOI: 10.32473/flairs.v35i.130584.

Appendix A. Evidence sources and reproducibility

The sources below are project evidence, separate from scholarly references. Relative links resolve in the repository checkout. The historical baseline and later packages retain their respective source identities. The current claim-to-source map records support and limits.

S0 — Accounting validity. E0 report and evidence. Full evidence commit 29d8945862b7df1bd5d7879ba1d980a388f6be0d.

S1 — Waiting-room capacity. E1 report, with five-draw comparison and validation records.

S2 — Exploratory admission/placement decomposition. E2b report and factorial evidence.

S3 — Incident replication and reversal. E2c report and E2d report. Frozen E2d evaluator commit 2f63706f46319433a2ba3af1df97afd0e56a95d1; exact E2c controls reused.

S4 — Experimental architecture and implementation. Frozen method, per-task source, evaluator source, environment source, and implementation correspondence, PDF pages 1–2 for Putra’s confirmations.

S5 — State-information sensitivity. Timing contract, five-cell comparison and report-state audit.

S6 — Forwarding sensitivity. Qualification, outcomes and analysis validation.

S7 — Morning exploratory pilot. Results and canonical input validation.

S8 — Morning primary replication. Protocol, manifest, paired differences and paired intervals. Full follow-up evaluator commit 908bd10f86542de94fc38af90dd56c2ccc08cf9b.

S9 — Five-RSU mechanism audit. Completed post-hoc report and audit results with input identities.

S10 — Unrun falsification proposal. Rotating-tie-break prefix protocol.

S11 — Artifact and preservation. Product architecture, release/execution boundary, evidence archive inventory and compact verification receipt. The separate E3 Dynamic Resource V2 campaign remains unexecuted.

S12 — Contribution and repair history. Read-only inspection of `vec_env` commits 4cb7c06, 0f01f4d and 2f63706, and TrafficTwin E0 validation tests at 29d894. Source comments explicitly credit my questions; my 15 September confirmation records my design, implementation changes and completed verification.

S13 — Retrospective gap closure, 8 September 2026. The verification entry point documents the field contract, source bindings, 87-file authentication, 19-run audit, 23 paired transitions, 440-case reference and production comparison. These completed checks and their historical sources remain unchanged.

S14 — Production-domain mechanism analysis. Model and proof, initial diagnostic protocol, bounded follow-up protocol, executable diagnostics, full results, numerical follow-up, adverse example and helper checks, dimension checks and outcome accounting. This is retrospective analysis added after the completed experiments and gap closure.

S15 — Retrospective empirical/engineering extension. Declared analysis protocol, task-type counts and bindings, benchmark implementation and measured results, cyclic implementation, compatibility evidence, sealed confirmation protocol, dry-run receipt, and verification route. That original preparation package remains disabled and unchanged; S16 records the separately authorised execution.

S16 — Authorised joint-randomness confirmation. Execution amendment, qualified execution protocol, execution seal, versioned evaluator, runner and completion gate, task/queue/block validation, sealed analysis, qualification receipt, prelaunch safeguard checks, and verification/access route. Completed primary analysis, all cell counts, block controls and raw inventory bind the new results. This study is separate from the original four-draw sample, pilot and retrospective analyses.

Table A1 records the execution environment. Historical manifests retain their own identities. Full reproduction also needs the frozen actor, traces and authorised inputs.

Table A1: Execution environment. Research runtime values come from the benchmark and confirmation receipts; the compact verifier and product dependencies are identified separately.

Component	Recorded value
Hardware chip	Apple M5 (CPU)
CPU count	10, as recorded by the qualification receipt
Physical memory	not recorded
Operating system	macOS 26.4.1, arm64
CPython (evaluator)	3.11.15
JAX / JAXlib	0.4.30 / 0.4.30
NumPy (evaluator)	1.26.4
SciPy (compact verifier)	1.17.1; see Appendix G and its command receipt
Streamlit (product lockfile)	1.59.2 at source base 1e01b75
SUMO	1.27.0, as reported by the source study [10]
Frozen evaluators	2f63706; 908bd10; versioned confirmation copy: joint_confirmation_2026-09-08/experimental/evaluator_v2.py (S16)
JAX x64	disabled

S17 — Examiner-revision artefact evidence. The source inventory, compact recomputation, descriptive split, figure inputs and revision validation are generated from the pinned repository without new research execution. The revision record distinguishes historical receipts, document checks, the platform workflow capture and author-only completion items. The research-software source base cited here is 1e01b755b8b633f43c9c7bb6fdd0d75beb6469e8. The real-data capture receipt and adapter validation receipt bind Figures 7 and 8. The platform exposes the same services through the `traffictwin` integration to `command-line` group (`validate`, `matrix`, `compare-campaigns`, `provenance`, `audit`, `report`). The full-run inventory counts 274 distinct full-run records, counting the E0 reference reused by E1 once; 26 historical records establish full-run identity through accepted campaign receipts and manifests rather than retained per-summary horizons.

S18 — Declared follow-up studies (15 September 2026). `PROTOCOL.md`, `ANALYSIS.json`, `CELL_RESULTS.csv`, `PAIRED_EFFECTS.csv`, `FINAL_ANALYSIS_AUDIT.json`, `TIMINGS.json`. The study completed 72 full cells, with 0 failures, in 3 h 55 min; the protocol was sealed before outcomes.

S19 — Three-trace generalisation study (16 September 2026). PROTOCOL.md, TRACE_INVENTORY.json, ANALYSIS.json, CELL_RESULTS.csv, PAIRED_EFFECTS.csv, FINAL_ANALYSIS_AUDIT.json. PR #144 remains open; this source cites the study at c95e4f8 on research/dissertation-traces-2026-09-16. Archive registration and main integration are deferred until after submission. The study completed 120 full cells, 18 qualification attempts and 24 block receipts, with 0 failures; clock time was 9 h 34 min including a 2 h 12 min host sleep; trace selection and the legacy-convention caveat are recorded in the inventory.

Table A2: Research and product artefact inventory at source commit 1e01b75. Sizes count physical Python source lines, including comments and blank lines. Research coverage refers to archived receipts unless explicitly labelled as a new document check [S17]. The role and required conditions/checks identify each component’s contribution; these include task-handling rules, evidence acceptance and software verification. Size alone proves neither quality nor independent authorship.

Component	Size	Tests / recorded checks	Required conditions / checks	Role
Evaluator	1,204 lines; versioned confirmation copy 1,331	E0 negative fixture; archived compatibility and 32 cell receipts	Admission, outcomes and service conservation	Computes the experimental task population and outcomes
Per-task selector	169 lines	Reference/helper and restart checks in S13–S16	Immediate reservation; one final commitment	Sequential least-workload dispatch
Cyclic comparator	46 lines	Pointer/restart and unchanged-arm compatibility	Persistent pointer; no retry or full-node skipping	Workload-unaware causal control
Campaign runner	138 lines	Prelaunch safeguards; 32 completed cells in S16	Source, seeds, completion and validation bindings	Executes and records the declared campaign
Campaign validators	160 lines plus compact verifier	8 block receipts; 32-cell compact recomputation	Reject inconsistent inputs, counts and identities	Separates successful execution from admissible evidence
TrafficTwin product	678 Python files; 270,721 lines	Adapter validation and capture receipts (S17)	Configured Ruff, mypy, pytest, archive and fixture checks	Streamlit, CLI, typed services and provenance workflows

Repository software-testing records are retained separately from the research-campaign validation records.

Appendix B. Audit coverage and diagnostic boundaries

The September audit independently identifies offers from active masks, V2I admission from enqueue fields and non-V2I failures from rejection predicates, then reconciles flags, categories, latency/deadlines, assignments and summaries.

Task joins require matched trace-row/substep/slot coordinates and corresponding fleet, task, arrival, action and ingress fields. Unsaved sizes correspond through the arm-independent key schedule. Gains minus losses must equal the success difference over the common offered denominator; later queues can diverge, so these are descriptive transitions.

Retrospective diagnostics distinguish simultaneous common-target admission A, causal admission replaying A's targets B, and causal reselection C. The completed proof and numerical investigation are retained in Appendix C, separately from observed fleet results [S14].

Consider a constructed explanation, not an observed historical pair: one queue place remains and two viable tasks each require 40 ms. Legacy fractional enqueue adds $(1/2) \times 80 = 40$ ms, although both can remain in the scored population. With unequal services, fractional enqueue can also differ from the work of the particular admitted task. Reject-mode admission instead accepts one task in order, enqueues its full 40 ms and assigns the other a failure.

The relevant invariant is that offers partition into admissions and non-admissions, with no execution or success assigned to rejected work. E0's negative fixture changes the summary offered count without adding a task record and must fail. Summary work partition alone is weaker: rejected work is calculated as offered minus admitted. Later reconstruction instead checks enqueue, drain and queue endpoints independently.

Table B1 retains the completed gap-closure audit coverage. Those raw checks and joins were not repeated for this revision.

Table B1: Historical availability and retained gap-closure audit coverage, 8 September 2026. Historical receipts remain unchanged. Archived summaries and raw-array checks provide different coverage; historical scoring impact remains unquantifiable.

Records	Available evidence / coverage	Completed gap-closure checks	Consequence
E0: two ten-step repeats and full seed 0	Surviving receipts, manifests, sources; full receipt records 834,120 cap rejections	Not assessable from available records	Raw arrays not retained; summaries and receipts archived
E1: five seeds $\times$ three caps	Fifteen run summaries/receipts; seed-0 middle cap reuses E0	Not assessable from available records	Original interval reproduced from summaries; scoring impact unresolved
E2 off/jsq/dla seed 0 and E2b reused cells	Hash-bound summaries and receipts; E2b reuses E2 exactly	Not assessable from available records	No new cell or historical task join inferred from reuse
E2b new ingress / E2c / E2d originals	Compact sources/records survive; original raw availability not established	Not assessable from available records	No non-retention or fresh raw-validation claim extended to these studies

Records	Available evidence / coverage	Completed gap-closure checks	Consequence
September morning primary	Twelve full runs, seeds 0/2/3/4; 1,744,116 offered per cell	Zero discrepancies in each inspected run	Supports these gate-enabled records; excludes pilot from inference
September morning pilot	Two full seed-1 runs, 1,744,116 offered each	Zero discrepancies in each inspected run	Remains exploratory, outside primary inference
September incident sensitivity pilot	Five full seed-1 runs, 13,076,234 offered each	Zero discrepancies in each inspected run	Does not replace historical E2d records
Other September arrays	Remaining inventory entries authenticated only	Integrity-only in the prior audit	Qualification/preflight files are not additional fleet replications

The per-run JSON records preserve offered/admitted counts, explicit rejection and unavailability categories, false-success and latency discrepancy counts, penalty-equal ambiguity, summary reconciliation and source bindings. A rejected task’s finite latency is not necessarily a false success. For the 19 available full runs, both false-success and non-penalty rejection-latency counts are zero, and fixed-history admission-consistent scores equal their archived values. Historical unavailable cells are labelled “not assessable,” never zero. Receipt checks are reported as past observations, not freshly rerun validation.

Table B2 details primary morning ingress→per-task task accounting. “Both” columns partition the same offered population; no task-level confidence intervals are constructed.

Table B2: Descriptive task transitions from ingress to per-task placement; four primary morning draws, pilot excluded. Within each draw, gains minus losses give the net change over the same offered tasks.

Fleet seed	Failure → success	Success → failure	Success both	Failure both	Net successes
0	71,152	3,340	1,545,724	123,900	67,812
2	83,543	4,640	1,522,085	133,848	78,903
3	70,862	2,913	1,550,789	119,552	67,949
4	59,264	1,949	1,569,719	113,184	57,315

All 23 eligible within-study pairs are retained in the machine-readable record. Task identity uses matched trace-row/substep/slot coordinates and verified stream identities; sampled sizes are source-derived correspondence, not independently observed fields. Later queue divergence prevents these descriptive transitions from identifying a unique causal contribution of individual placements.

The scalar suite includes 440 primary cases and ten deletion trials, below its 500-case limit. The five-task case is minimal under single-candidate deletion, not a globally minimal counterexample over every numeric parameter. A’s retained mask remains unchanged by the diagnostic code: the fourth replacement is inspected only. C’s repeated complete passes restart from the same state. That scalar exercise performed no full-evaluator prefix, actor inference, SUMO run, campaign or retraining; the new authorised campaign is separate.

E1 varied waiting-room capacity across five matched draws. The archived score difference for 99,520 versus 1,866 tasks per RSU was -0.01094 percentage points, with a 95% interval $[-0.02465, +0.00276]$. Recalculation from surviving run summaries reproduces this

interval; it is not fresh task-level validation. The archived E1 score contrast is statistically inconclusive under the historical implementation. Its interpretation as admission-consistent deadline attainment remains qualified by the unquantified scoring-mask exception. The interval describes sampling uncertainty conditional on that implementation, not the additional measurement uncertainty. Neither equivalence nor statistically supported degradation follows [S1].

The saved secondary summaries report more admissions and lower rejection at larger limits, while mean admitted latency rises from approximately 3.77 to 12.19 and 132.49 seconds. These changing admitted populations cannot substitute for the offered-task contrast; their latency summaries retain the scoring-mask qualification. More retained work need not mean more useful service.

Changing the off scoring mask could affect transmit energy, SoC and later observations: even admission-consistent saved flags would describe fixed history, not a corrected rerun. Separately, the completed audit found no non-admitted successes or non-penalty rejection latencies in 19 authenticated September gate-enabled runs, with admission/category/summary agreement. It supports those outcomes without resolving historical measurement impact.

Appendix C. Exact-model result and numerical boundary

This appendix retains the retrospective exact-model argument, numerical boundary evidence and adverse example. These are distinct from the sampled fleet contrasts. Sources S13–S15 bind the completed mathematical and executable diagnostics; the Declaration records assistance.

C.1 Model and objects that must remain separate

Candidates have a fixed order $i = 1, \dots, n$ and fixed destination r_i . At each destination r , initial workload $W_r \geq 0$, integer task count $Q_r \geq 0$ and capacity C_r are fixed throughout reconciliation. E_i is fixed active/radio/coarse-capacity eligibility. Candidate service $s_i \geq 0$ and deadline D_i are finite. There is no intervening service drain. All sums here are exact.

Let $E, M \in \{0, 1\}^n$ with $M_i \leq E_i$. Workloads W, O, s and deadlines D are in milliseconds; Q, H, C count tasks. Define $O_i(M) = \sum_{\substack{j < i \\ r_j = r_i}} s_j M_j$ and $H_i(M) = \sum_{\substack{j < i \\ r_j = r_i}} M_j$. The map is

$$F_i(M) = E_i \mathbf{1}\{W_{r_i} + O_i(M) < D_i\} \mathbf{1}\{Q_{r_i} + H_i(M) < C_{r_i}\}. \quad (\text{C1})$$

Start $M^0 = E$; the historical algorithm retains $M^3 = F^3(E)$, then recomputes $O(M^3)$. Its gate-failure label is $E_i \mathbf{1}\{W + O_i(M^3) \geq D_i\}$; its capacity label covers other eligible non-admissions, even when the retained mask is not fixed. Enqueueing follows M^3 . Diagnostic success is $M_i^3 \mathbf{1}\{W + O_i(M^3) + s_i \leq D_i\}$, with zero transfer latency for these fixtures. Production latency additionally contains the source-defined communication/forwarding terms. Neither admission nor a rejection label alone determines the final deadline outcome.

The causal fixed-target scan computes $b_i = F_i(b)$ in order, reserving s_i and one count immediately when admitted. It uses the same E , destinations and input arrays. This scan is B in a one-substep comparison. Across multiple substeps B replays A's entire target sequence; it is a target-conditioned diagnostic, not automatically an autonomous deployment policy.

C.2 Existence, uniqueness and candidate-count bound

Proof. F_i depends only on earlier candidates at the same destination. Define b_1 from its constant predicate, then b_2 from b_1 , and so on. This constructs a fixed point. If two fixed points differed, their earliest differing candidate would see identical preceding inputs and hence could not differ. Thus b is unique and is exactly the causal scan.

From any initial binary mask $M^0 \leq E$, after t simultaneous replacements the first t eligible candidates at each fixed destination have their final values: induction uses the already-correct preceding values. Therefore at most n_r replacements suffice at destination r , and at most $\max_r n_r$ globally. Ineligible candidates can be removed because their mask is fixed at zero. Multiple fixed destinations form disjoint dependency chains. This argument alone does not justify three replacements for an arbitrary candidate population.

□

C.3 A deadline-count bound from eligibility

Let d_r be the number of distinct deadline thresholds among eligible candidates at destination r . Starting at E , exact reconciliation reaches b in at most $\min(n_r, 2d_r - 1)$ replacements per nonempty destination. Empty destinations need none. Nonnegative work makes F order-reversing: $U \geq V$ implies $F(U) \leq F(V)$. Since $E \geq b$, odd iterates are lower bounds on b and even iterates are upper bounds. We prove the stronger statement that any upper mask $b \leq U \leq E$ reaches b within $2d - 1$ replacements at one destination.

Proof. If $U = b$ there is nothing to prove. Otherwise let p be its earliest extra admission: $U_p = 1, b_p = 0$. All earlier entries coincide with b , so after one replacement the entire prefix through p is correct and remains correct. If p is rejected by capacity, that fixed earlier prefix already fills capacity, and every later candidate is rejected; $F(U) = b$.

Otherwise p is gate-rejected and the fixed earlier workload is at least D_p . Every later candidate with deadline $\leq D_p$ must then be rejected, because subsequent work is nonnegative. These entries are zero after the first replacement and stay zero. After two replacements, $F^2(U)$ is again an upper bound on b , with that prefix and those later rejected classes fixed. Delete those fixed entries and absorb the fixed prefix into W and Q . The remaining suffix has at most $d - 1$ distinct deadlines, unchanged order and the same form of gate/capacity rule. Induction needs at most $2(d - 1) - 1$ further replacements, giving $2d - 1$ overall. For $d = 1$, the first gate rejection excludes every later candidate, so one replacement suffices. Capacity can only terminate the suffix earlier.

□

Zero work is allowed: it need not advance workload, but every admission still advances count. Initial workload at/above a deadline, capacity already full, ineligible gaps and arbitrary ordering of the two thresholds do not defeat the proof. The same argument also gives one replacement when deadlines are nonincreasing in candidate order: the first extra gate rejection excludes every subsequent deadline class. The bound is sufficient; it need not be attained for a particular input.

TrafficTwin's distinct production deadlines are 100 and 500 ms, so **three replacements equal the causal fixed-target mask in exact arithmetic** under these assumptions. Its service distributions are positive and do not weaken this conclusion; no independent choice of a convenient service/deadline is needed for the proof. Ingress with fixed individual targets also decomposes by destination. Causal per-task reselection does not have fixed targets, so this result does not equate A with C.

The earlier five-task 1/1/41/41/81 example has three deadline thresholds and lies outside this two-threshold result. Its instability is therefore not evidence for an exact-arithmetic production-domain reconciliation defect. The new deduction explains why the earlier two-threshold grid was stable without using that grid as proof.

C.4 Why float32 requires a separate claim

The frozen helper computes an inclusive `jnp.cumsum(w, axis=0)` and subtracts w to form the exclusive prefix. Exact cancellation removes the current candidate; rounded addition/subtraction need not do so identically for both values of its own mask. It also adds base workload after prefix accumulation. The causal helper starts at base workload and

adds admitted services in scan order; floating-point associativity differs. Thus the exact dependency and upper/lower arguments are not a proof of float32 equivalence or convergence.

Count ranks use float32 cumulative sums too. Integer counts within the small suite and the studied slot widths are exactly representable, but that does not certify arbitrarily large populations. Strict gate comparisons require exact mask comparison, not an allclose acceptance criterion. The diagnostic protocol declares numerical reporting tolerances before execution, retains all discrete disagreements and separates task-parameter compatibility, whole-model reachability and observed frequency. No saved September candidate masks establish how often any constructed threshold case occurs.

C.5 Consequence for queue clearing in the stated exact model

With service multiplier 1, the largest possible RSU task service is less than 38.889 ms (type2, nominal 35.354 ms times noise below 1.1). At a destination with admissions, the last admitted task sees less than its deadline, at most 500 ms, before adding its own service. The final accumulated workload is therefore less than 538.889 ms, provided initial workload does not already exceed this bound. If nothing is admitted, workload is unchanged. Starting empty, induction across substeps preserves the bound for either fixed-target causal admission or causal reselection. The exact two-deadline result transfers it to three-pass fixed-target reconciliation.

One 1,000 ms drain consequently clears every queue from this initialisation. Thus the empty-boundary assumption in the earlier destination-prefix explanation follows from the stated exact gate/service/timing model, not traffic sparsity alone. The observed zero end-points and reconstructed maxima near 538 ms are consistent with that consequence. This is not a float32 error bound, a validation of event-level service or a theorem for other deadlines/service multipliers/drain intervals. Together with $R \geq K$, positive productive admissions and lowest-index ties, it supports recurrence of the low-index destination prefix; it does not identify every deadline loss.

C.6 Retained diagnostic evidence

The initial 671 inputs comprise 128 general mathematical fixtures, 324 coupled fixtures, 210 deliberate threshold fixtures and nine completion boundaries. Bounded adverse reductions/transfers and numerical follow-ups bring the total to 719, within the predeclared ceiling of 800. Protocol amendments precede their specific follow-ups; no outcome-driven expansion beyond that budget occurred. Full negative and unchanged results remain in the machine-readable record.

The numerical alternating case uses seven type3 services of 2.5252525806427 ms followed by one type1 service of 21.11111068725586 ms, all with 100 ms deadlines. Initial $W = 82.32322692871094$ ms and $Q = 4$, $C = 6,220$. Four earlier type1 services of $W/4$ construct this queue in a separate local admission substep; their noise multiplier lies within 0.9–1.1. This establishes local kernel compatibility under explicit arrays, not exact RNG/actor/traffic reachability.

Exact arithmetic admits all eight, with the last gate margin about 0.000005007 ms; the last task nevertheless misses completion. The instrumented float32 mirror alternates

11111111 $\rightarrow$ 11111110 $\rightarrow$ 11111111 $\rightarrow$ 11111110 $\rightarrow$ 11111111. The unchanged production helper confirms the retained third mask, recomputed offsets and capacity-failure count. Its causal float32 scan also rejects the last candidate, though via its own rounded gate. Thus this fixture distinguishes exact from implemented arithmetic and exposes an unstable label, not a demonstrated A $\rightarrow$ B success improvement. The mirror’s intermediate masks are constructed diagnostics, not recorded September fields.

Four padding checks retain the alternating mask at $N = 215/R = 9$ and $N = 2,488/R = 10$, with the active block at either end. Their target is imposed: a busy target would not be the common argmin while others are empty. A final fixture prepends the four services, starts every queue empty and selects RSU0 at the correct $N = 215/R = 9$ dimensions. Its third mask is stable, although the last admission differs from exact arithmetic. This comparison prevents treating fixed-target instability as demonstrated on a full common-target trajectory.

One separate inclusive-completion boundary also differs: saved-offset arithmetic rounds a just-late exact completion to an on-time float32 value. This is a zero-transfer calculation, not execution of the full latency scorer. Neither numerical example measures historical frequency or changes the reported fleet estimates.

The two-RSU adverse case is minimal under the tested shared-position deletions and one-substep reduction. It retains production task/deadline/capacity values but changes infrastructure dimensions; its twelve transfers to the studied RSU/substep parameters retain no adverse result. No fixture frequency is interpreted as a fleet probability; that numerical investigation contains no confidence intervals, raw audits or full-system runs.

C.7 Retained adverse dispatch illustration

Target reselection can separately lose useful completions. Table C1 uses two empty RSUs, two substeps, capacity 6,220, zero transfer latency and repeated type2/type2/type1/type2 tasks with nominal production service demands. Both arms admit all eight tasks. B replays A’s targets 0 then 1, whereas C chooses the current least workload after every reservation.

Table C1: Reduced-infrastructure illustration with production task/deadline/capacity values. Every listed task is admitted in both arms; completion is diagnostic service latency in milliseconds. “Slot.task” preserves candidate order. Values shown to three decimals; tests retain full precision. The later short-deadline task illustrates the lost completion.

Slot.task	Deadline	Service	B: RSU / work before	B: completion	C: RSU / work before	C: completion
1.1	500	35.354	0 / 0.000	35.354 met	0 / 0.000	35.354 met
1.2	500	35.354	0 / 35.354	70.707 met	1 / 0.000	35.354 met
1.3	100	21.111	0 / 70.707	91.818 met	0 / 35.354	56.465 met
1.4	500	35.354	0 / 91.818	127.172 met	1 / 35.354	70.707 met
2.1	500	35.354	1 / 0.000	35.354 met	0 / 56.465	91.818 met
2.2	500	35.354	1 / 35.354	70.707 met	1 / 70.707	106.061 met
2.3	100	21.111	1 / 70.707	91.818 met	0 / 91.818	112.929 miss
2.4	500	35.354	1 / 91.818	127.172 met	1 / 106.061	141.414 met

B preserves an empty RSU for substep 2. C spreads earlier work across both, making the later 100 ms task finish at 112.929 ms: eight admissions produce seven successes. Current workload omits future deadlines, and the gate omits own service; neither directly maximises deadline-success count. Tested deletions remove this loss, without establishing global minimality.

The loss did not persist in twelve tested transfers to nine/ten RSUs and five substeps, with four/six candidates. The example identifies an order/service/deadline interaction, not the cause of recorded losses at full dimensions. B replays A targets and is not an autonomous deployable comparator. Original full-evaluator prefixes remain unrun [S10], [S14].

C.8 Reconstruction and task-outcome details

Original morning common-target work used only RSUs 0–4, with mean shares approximately 48.34%, 30.33%, 14.63%, 5.21% and 1.48%; RSUs 5–8 received none. Per-task shares were close to one ninth each. Figure 9 describes allocation, without establishing that equal workloads caused the attainment difference [S8].

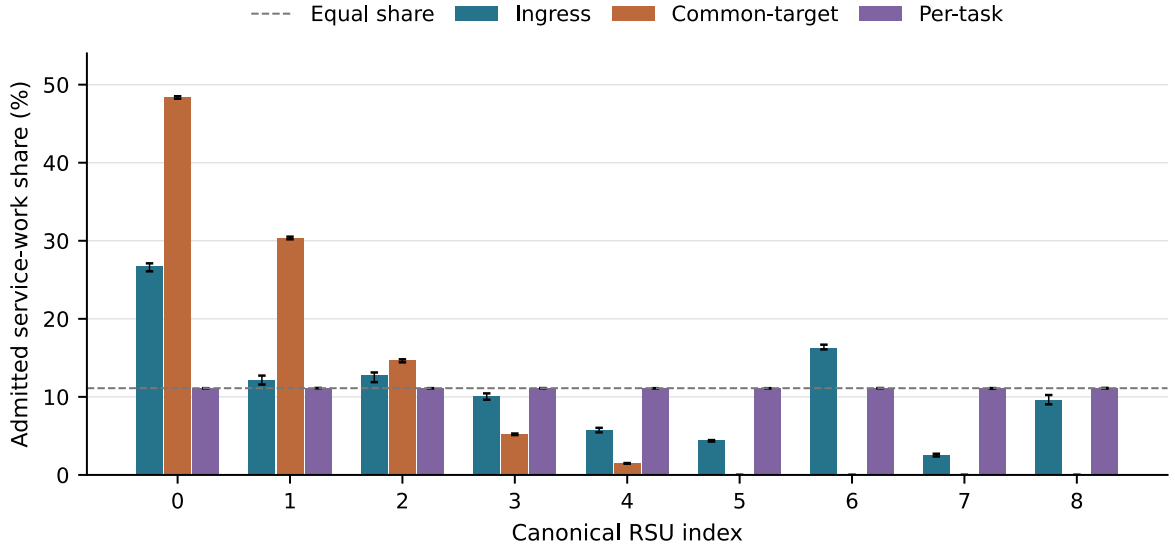


Figure 9: Admitted service-work distribution, redrawn from archived CSVs. Bars are means; whiskers are observed minima/maxima over four fleet draws, not confidence intervals. Aggregate shares motivated a separate post-hoc audit; they do not by themselves establish intermediate scheduling decisions or counterfactual task outcomes.

The post-hoc audit found zero saved end-of-second workload and task counts in all four common-target runs, and target k at substep k whenever a V2I attempt exposed it. Empty substeps do not reveal the internal selector. Intermediate workloads were reconstructed, not saved directly, using recorded admissions/destinations and the qualified service reference. All endpoints and observable target minima agreed, although zero endpoints alone weakly constrain intermediate paths. Reconstructed pre-drain workload reached approximately 538.730 ms; task counts stayed below 26, far from the 1,000 ms drain and 6,220-task ceiling [S9].

The per-batch bound in Section 3.5 follows because each of K substeps selects at most one of R destinations: at most $\min(K, R)$ receive new assignments. It requires neither empty

queues nor a particular tie rule and concerns new assignments, rather than work already present.

A recurring low-index prefix requires the additional conditions discussed in Section C.5: empty batch starts, positive productive admissions, lowest-index ties, no intervening drain and $R \geq K$. Starting at RSU 0, productive substeps advance through empty indices; unproductive ones do not. End-of-batch clearing resets the tie. Without clearing or deterministic ties, the same per-batch bound could hold while the full-run union covered every RSU.

The arrays recorded 511,684 deadline-gate rejections and zero RSU-capacity rejections. In reconstructed states, every gate rejection coexisted with at least five idle RSUs possessing spare task capacity. Larger waiting rooms would not address that gate bottleneck. Spare capacity does not establish that each task would succeed elsewhere: redistribution changes later queues, and end-to-end latency also matters.

Sections C.2–C.3 prove the fixed-target result, and Section C.5 derives queue clearing under the exact gate/service/timing assumptions. Here “production” refers to the studied evaluator configuration, not deployed roadside hardware [S14].

The float32 caveat in Sections C.4 and C.6 also applies to this reconstruction: different accumulation orders can change a strict gate even for sub-0.001 ms differences [43]. The constructed diagnostics establish neither full-trajectory reachability nor historical prevalence.

The adverse illustration in Section C.7 and Table C1 shows that reselection can lose completions under reduced infrastructure. Its unsuccessful transfers to the studied dimensions do not explain the cause or frequency of recorded losses. The proposed full-evaluator prefixes remain unrun [S10], [S14].

Task outcomes in the original morning sample.

The mechanism explains destination concentration, but does not uniquely allocate the performance gap to individual decisions. The following recorded accounting locates gains and losses in the original four-draw morning comparison, separately from the eight-block confirmation.

Table C2 partitions gains and losses over the same 1,744,116 offered tasks per original morning draw: 284,821 gains – 12,842 losses = 271,979 net successes. Previously gate-rejected tasks supply most gains, alongside recovered admitted misses; both kinds of losses remain visible. Later queues differ, so these are descriptive transitions, not individual-decision causal effects. The four draws remain the replications [S13], [S14].

Table C2: Morning ingress→per-task outcome accounting, re-extracted from the authenticated paired results. Gains become successes; losses cease to be successes. All four primary draws are retained; the pilot is excluded. Gains minus losses equal the matched success difference.

Seed	Gain: gate rejection	Gain: admitted miss	Loss: non-admitted	Loss: admitted miss	Net successes
0	53,061	18,091	395	2,945	67,812
2	64,418	19,125	496	4,144	78,903
3	52,432	18,430	105	2,808	67,949
4	41,823	17,441	48	1,901	57,315
Total	211,734	73,087	1,044	11,798	271,979

Table C3: Retrospective type-level net additional successes, per-task minus ingress. Each type has the stated offered population in every primary draw; full successes, admissions and failure counts are supplied in Appendix D. Types are subgroups within a draw, not additional independent replications.

Type / deadline	Offered per draw	Seed 0	Seed 2	Seed 3	Seed 4
1 / 100 ms	349,045	+19,823	+19,165	+20,339	+20,254
2 / 500 ms	523,060	+72	+62	+6	+5
3 / 100 ms	872,011	+47,917	+59,676	+47,604	+37,056

Table C3 locates 79,581 net successes in Type 1, 145 in Type 2 and 192,253 in Type 3. Type 3 supplies 70.7% of the net gain. Within-type improvements span 5.491–5.827, 0.001–0.014 and 4.249–6.843 points, respectively. The two 100 ms types differ substantially in payload and service (§ 2.3). No type loses net successes in any draw, but individual losses persist. In draw 2, Type 1 gate rejections fall by 23,152 while admitted misses rise by 3,987. This supports reduced rejection as an important contributor, without isolating deadlines from communication/service demand or identifying individual-placement causal effects [S15].

Appendix D. Type aggregation and engineering evidence

Scheduler timing uses actual JAX helpers, including three-replacement prefixes; precomputed candidate work replaces only the service provider. Sixteen configurations combine four arms, $N/R = 215/9$ or $2,488/10$, $K = 5$ and sparse/empty or busy/nonempty fixtures. Arms share arrays with coupled type/deadline/service values. Busy entry queues are imposed stress states, excluded by exact clearing from empty initialisation (§ 3.5). Following JAX guidance, compilation is separate, inputs are device-resident, five warm-ups precede 100 synchronised calls. Actor, radio, service generation, transfer, scoring and I/O are excluded. Repetitions measure host variability, not fleet uncertainty [42]. Fixed order and uncontrolled frequency/thermal state limit timing comparisons; durations and IQRs remain available.

Compiler buffer reuse and elimination of unused broadcasts or identical passes affect execution, so asymptotic work alone does not predict the measured ordering. Three vectorised replacements materialise $N \times R$ arrays: $O(KNR)$ work and $O(NR)$ intermediate space with parallel prefix dependence. Per-task also requires $O(KNR)$ work for R -way minima, but has a $K \times N$ sequential dependency chain. Round-robin removes minima, giving $O(KN)$ target/predicate work in an indexed-state model. Both retain R queue entries and emit $O(KN)$ diagnostics.

The retained retrospective type analysis read eight original primary-morning task arrays, authenticated their hashes, and aggregated categories by type; it was not repeated here. The prior 19-run admission validation and 23 task joins are reused, not repeated. All-task totals and paired net changes reconcile. The following table exposes every type/draw/arm; attainment uses the offered column. Other non-admissions are available in the linked CSV and unchanged within each pair.

Table D1: Type outcomes. I = ingress, P = per-task; offered and category counts are tasks, attainment is percent. Gate rejection and admitted misses are disjoint. Each task type retains its own offered denominator.

Seed / type	Arm	Offered	Success	Attainment	Admitted	Gate rejected	Admitted miss
0/1	I	349,045	252,283	72.278	328,923	19,548	76,640
0/1	P	349,045	272,106	77.957	347,605	866	75,499
0/2	I	523,060	516,034	98.657	522,134	46	6,100
0/2	P	523,060	516,106	98.671	522,180	0	6,074
0/3	I	872,011	780,747	89.534	821,868	48,572	41,121
0/3	P	872,011	828,664	95.029	868,284	2,156	39,620
2/1	I	349,045	242,629	69.512	324,165	24,149	81,536
2/1	P	349,045	261,794	75.003	347,317	997	85,523
2/2	I	523,060	515,409	98.537	521,901	40	6,492
2/2	P	523,060	515,471	98.549	521,941	0	6,470
2/3	I	872,011	768,687	88.151	809,915	60,362	41,228
2/3	P	872,011	828,363	94.995	867,771	2,506	39,408
3/1	I	349,045	253,921	72.747	330,364	18,383	76,443
3/1	P	349,045	274,260	78.574	348,537	210	74,277
3/2	I	523,060	517,080	98.857	522,533	1	5,453
3/2	P	523,060	517,086	98.858	522,534	0	5,448
3/3	I	872,011	782,701	89.758	824,760	46,357	42,059
3/3	P	872,011	830,305	95.217	870,539	578	40,234

Seed / type	Arm	Offered	Success	Attainment	Admitted	Gate re- jected	Admitted miss
4/1	I	349,045	260,449	74.618	333,800	14,596	73,351
4/1	P	349,045	280,703	80.420	348,250	146	67,547
4/2	I	523,060	516,064	98.662	521,928	1	5,864
4/2	P	523,060	516,069	98.663	521,929	0	5,860
4/3	I	872,011	795,155	91.186	834,388	35,739	39,233
4/3	P	872,011	832,211	95.436	869,818	309	37,607

The benchmark record retains lowering/compilation times, median, quartiles, p95, minima/maxima, compiler buffer estimates and exact source/runtime identities for all 16 configurations. Raw call durations permit descriptive timing checks. Precision sensitivity records pre-outcome assumptions about block variance; it is not observed power for the new results.

Table D2: Scheduler-only engineering trade-off in the environment recorded in Table A1, float32, default runtime threads. Times are milliseconds per five-substep batch; *S/B* denote sparse/busy fixtures. Temporary memory is XLA buffer analysis, not peak RSS. Scheduler cost is separate from simulated task latency.

Policy	$N = 215$ median <i>S/B</i>	$N = 2,488$ median <i>S/B</i>	Busy p95, 215/2,488	Temporary KiB, 215/2,488
Ingress	1.031/1.019	6.096/6.113	1.103/6.230	24.6/294.9
Common-target	0.952/0.962	5.797/5.815	1.014/5.935	23.8/285.2
Per-task	0.052/0.055	0.501/0.616	0.062/0.641	11.7/139.1
Round-robin	0.025/0.021	0.223/0.186	0.023/0.201	11.7/139.1

Appendix E. Joint-randomness study controls and execution

The execution-authorized seal preserves the disabled preparation package and binds its own source version, current actor/trace identities, runtime, output root, lock and qualification receipts. Fleet/evaluator pairs are (100,200) through (107,207). The accessible manifest check found no prior outcomes for these pairs; this is a scoped absence check, not proof that no record exists elsewhere. Block 0 starts ingress/common-target/per-task/round-robin; each subsequent block rotates that order by one position.

Qualification used the previously inspected fleet/evaluator pair (1,0), never confirmation outcomes. Three frozen/instrumented pairs ran for 300 steps, followed by round-robin at 300 and a fresh-process 150-step restart: eight attempts total. All 83 shared scientific fields matched exactly in each unchanged-arm pair. The 300-step cyclic run advanced its pointer 8,817 times, including 28 subsequently rejected attempts; its final pointer was 6. The 150-step run matched the longer run's prefix. No unavailable-radio, RSU-capacity or local-capacity failures occurred in these short records, so their end-to-end branches were not empirically exercised. Preserved kernel tests cover those branches separately. Qualification is neither an extra replication nor proof of full-horizon equivalence.

Added records expose final admission independently of success, actual local/peer service, queue counts, post-service workloads, SoC/energy and cyclic pointer state. Negative checks refused a corrupted offered count, mismatched shared-input binding, absent block receipt and changed output hash. A separate prelaunch critique identified a possible alternate-root attempt-budget bypass, incomplete type-summary checking and misleading failed-process timing. The final version binds one raw root/global lock, validates type numerators/denominators and records failure phase and elapsed process time. Existing short records were rechecked; no ninth qualification evaluator ran.

Each full cell requires exact discrete accounting, proposal/admission/execution/forwarding consistency, penalty latency exactly ten times the deadline, inclusive deadline success and independent queue-work reconstruction. Retained tolerances are 0.01 ms absolute per queue-second for service conservation, 0.001 ms plus relative 0.000001 for drain, and 0.000001 task for reconstructed JSON score/type numerators. Within each block, fleet arrays, key streams, offers, observation descriptors, operational types/sizes and pre-admission service work must agree exactly. Observations, logits, actions, SoC and selected radio quantities are separately reported; equality is not forced.

Execution is serial with at most 32 full attempts including failures, no retries or seed substitution, a three-hour per-cell timeout, 50 GiB initial free-space requirement and 20 GiB before each attempt. Incomplete attempts are retained. Completed-output hashes, commands, source identities and all eight block receipts gate the sealed primary analysis. The first included block supplies a remaining-runtime estimate; effects cannot change continuation. Process time includes startup, compilation, evaluation and compressed output writing; validation is timed separately. These times are not the scheduler-only benchmark in Appendix D, Table D2.

The completed full matrix has 32 valid cells, eight valid block receipts and no failed attempts. The following tables retain every cell and block. *I* = ingress, *D* = common-target, *P* = per-task and *R* = round-robin; fleet/evaluator seeds are 100+*b* and 200+*b* for block *b*.

Other nonadmission categories are retained in CELL_RESULTS.csv and each validation receipt.

Table E1: Full new-study counts. Offered populations match within blocks; attainment is offered-task percent. Gate rejection and admitted misses are disjoint. All eight paired blocks and four policies are retained; tasks are not pooled as replications.

Block / arm	Offered	Admitted	Successes	Attainment	Gate rejected	Admitted misses
0/I	1,744,761	1,655,779	1,525,913	87.457	86,625	129,866
0/D	1,744,761	1,582,636	1,459,384	83.644	159,768	123,252
0/P	1,744,761	1,737,398	1,605,152	91.998	5,006	132,246
0/R	1,744,761	1,726,560	1,592,830	91.292	15,844	133,730
1/I	1,740,970	1,662,805	1,534,040	88.114	76,351	128,765
1/D	1,740,970	1,593,599	1,470,819	84.483	145,557	122,780
1/P	1,740,970	1,736,301	1,607,919	92.358	2,855	128,382
1/R	1,740,970	1,727,104	1,596,377	91.695	12,052	130,727
2/I	1,741,855	1,639,136	1,510,719	86.730	99,922	128,417
2/D	1,741,855	1,556,771	1,436,166	82.450	182,287	120,605
2/P	1,741,855	1,732,694	1,598,164	91.751	6,364	134,530
2/R	1,741,855	1,718,789	1,583,772	90.924	20,269	135,017
3/I	1,744,227	1,677,045	1,557,192	89.277	65,368	119,853
3/D	1,744,227	1,616,965	1,501,691	86.095	125,448	115,274
3/P	1,744,227	1,740,631	1,624,157	93.116	1,782	116,474
3/R	1,744,227	1,734,468	1,614,712	92.575	7,945	119,756
4/I	1,743,484	1,680,263	1,560,221	89.489	61,226	120,042
4/D	1,743,484	1,623,533	1,507,624	86.472	117,956	115,909
4/P	1,743,484	1,738,717	1,622,888	93.083	2,772	115,829
4/R	1,743,484	1,732,685	1,613,895	92.567	8,804	118,790
5/I	1,745,078	1,685,944	1,564,006	89.624	56,788	121,938
5/D	1,745,078	1,629,177	1,511,177	86.597	113,555	118,000
5/P	1,745,078	1,740,768	1,623,996	93.062	1,964	116,772
5/R	1,745,078	1,735,213	1,614,934	92.542	7,519	120,279
6/I	1,742,095	1,669,339	1,541,879	88.507	69,142	127,460
6/D	1,742,095	1,603,847	1,481,655	85.050	134,634	122,192
6/P	1,742,095	1,736,400	1,611,494	92.503	2,081	124,906
6/R	1,742,095	1,729,087	1,601,138	91.909	9,394	127,949
7/I	1,744,742	1,657,846	1,531,566	87.782	84,523	126,280
7/D	1,744,742	1,588,286	1,468,334	84.158	154,083	119,952
7/P	1,744,742	1,736,713	1,608,728	92.204	5,656	127,985
7/R	1,744,742	1,726,431	1,596,904	91.527	15,938	129,527

Table E2: All eight paired effects, percentage points. These blocks, not tasks or RSUs, are the new replications.

Block	Per-task – ingress	Common-target – ingress	Per-task – round-robin
0	+4.542	−3.813	+0.706
1	+4.244	−3.631	+0.663
2	+5.020	−4.280	+0.826
3	+3.839	−3.182	+0.542
4	+3.594	−3.017	+0.516
5	+3.438	−3.027	+0.519
6	+3.996	−3.457	+0.594
7	+4.423	−3.624	+0.678

Actions matched exactly in all 24 non-reference arm/block comparisons. Small observation and logit differences occurred in block 1; they did not change actions. The estimand remains the total scheduling intervention under frozen weights. Observed action matching extends the original finding to these streams without forcing replay or guaranteeing future equality. In block 1 (fleet 101/evaluator 201), each alternative differed from ingress by at most 0.0000000149012 in observations and SoC, and 0.000000357628 in logits. These are dimensionless recorded differences; no action was altered or forced. The block receipts retain every comparison and exact maxima. The cell records and paired effects preserve unrounded values. No task-level significance tests or new type subgroup analysis were added.

All 32 records passed final-admission/score/category/penalty, assignment and queue checks under the sealed contract. Maximum reconstructed per-queue-second service discrepancy was 0.000121593 ms for RSUs and 0.000244141 ms for vehicle queues. This finite result does not certify every possible path or repair the historical off scorer.

The campaign elapsed 6440.25 seconds (107.34 minutes) from first attempt to completion. Full evaluator processes took 6375.54 seconds in total; cell validation took 49.59 seconds, block validation 10.83 seconds, and sealed analysis with completion/hash checks 3.44 seconds. Qualification processes separately took 68.09 seconds. Timing records and per-cell logs retain the exact boundaries. The first-block estimate used runtime only.

The raw inventory binds the locally retained task/step arrays and supporting files. The verifier (requires NumPy and SciPy) regenerates central tables from compact evidence and accepts an optional explicit raw root for integrity checking. Compact arithmetic is not repeated task-level validation; hashes are not a backup. The dated inventory documents local retention; it does not verify an off-machine copy.

Appendix F. Latency, scoring and drain details

The same archived one-hot-17 checkpoint is used in the initial studies; Section 3.4 declares the second-checkpoint condition. Its supplied Model C training used synthetic mobility, 128 environments, learning rate 0.003 and seed 100; training was not reproduced here. The provisional UK2030 preset samples RPi/Jetson/GPU tiers with probabilities (0.40, 0.35, 0.25) and EV status with probability 0.22, not measured Manchester shares. Actor training and trace remain fixed within each condition. Original replications vary fleet seed at evaluator seed 0; the later eight blocks vary both (§ 3.4).

V2V selection excludes the source and full-queue peers, then chooses the strongest remaining simulated link. Fading as well as distance determines quality, so this is not nearest-neighbour selection. Peer workload and capability affect latency, not this ranking. Observation and operational links use separate fading samples. Randy Prasetya Putra confirmed these inherited conventions and one mode/helper per vehicle-second; they were held constant across placement arms [S4].

For homogeneous RSUs, service milliseconds are $s = \frac{\xi \times C_t}{2.2 \times 1.5 \times \min(12, p_t) \times 0.9 \times \min(5, g_t)}$, where C_t is million cycles, $p_t = (4, 4, 2)$, $g_t = (40, 5, 1)$, and ξ is uniform on 0.9–1.1. The denominator uses 2.2 GHz, 1.5 instructions per cycle, effective cores, utilisation and capped GPU acceleration. Million-cycle/GHz conversion yields milliseconds. Before noise, services are approximately (21.111, 35.354, 2.525) ms. All initial placement arms use service multiplier 1; the half-speed follow-up is a separate condition (Section 3.4).

$T(x, q, b)$ is $0.1 + 1,000 \times 8x/b$ milliseconds for $q \geq 0.15$ and capacity $b > 0.000000001$ Mbps, with x in MB; otherwise it uses a 10^9 ms failure value (up to the 0.1 ms propagation addition). The 0.001 MB return term reuses the ingress quality and capacity. It is not a separately simulated downlink or inter-RSU route. The fixed forwarding charge f is added once. Local latency is vehicle backlog plus local service; V2V adds input/return transfers on the selected peer link, peer backlog, preceding local/V2V reservations and peer service. Service s derives from task cycles, device frequency, parallelism, utilisation and acceleration, with a sampled 0.9–1.1 multiplier. The placement code reserves the same sampled work used for latency: exact service knowledge is a declared information assumption.

The gate-enabled paths pass final admission into latency scoring. Historical `off` instead passes earlier coarse eligibility while enqueueing uses refined admission: an in-substep capacity rejection need not receive penalty latency. Its historical numerical impact remains unquantified (§ 3.1). The final mapping is $L[i] = 10D[i]$ when $L_{\text{raw}}[i] \geq 10^8$ ms, and $L[i] = L_{\text{raw}}[i]$ otherwise. Finite late latencies are not generally clipped; success uses $L[i] \leq D[i]$, inclusively. Finite rejection latency alone is not a false deadline success.

This aggregate carry approximation has no individual departure-event model; Equation (5) checks its declared dynamics rather than physical calibration. After the fifth substep, $S[r] = \min(W_{\text{post}}[r], 1,000)$ is served and $W_{\text{next}}[r] = W_{\text{post}}[r] - S[r]$. If this empties the queue, its task count becomes zero. Otherwise the evaluator subtracts $\lfloor Q_{\text{post}}[r] \times S[r] / W_{\text{post}}[r] \rfloor$, retaining at least one task while positive work remains.

A further 104 forwarded tasks had penalty-equal latency ambiguity but were already misses and remained misses under every nonnegative charge. Their latency ambiguity remains unresolved.

Four ten-step forwarding probes qualify fixed-latency transformations of saved full records: charges leave admission, queues and actions unchanged. No full positive-cost simulation or network model is implied; penalty-equal ambiguity remains unresolved [S6].

Appendix G. Artefact access and verification

The repository is <https://github.com/Abdulla4akash/traffictwin>. The manuscript branch is `docs/dissertation-examiner-revision-2026-09-11`, its source base is `1e01b75`, and the manuscript package is `docs/dissertation/examiner_revision_2026-09-11/`. The LaTeX source and its build inputs form this delivery.

Full reproduction depends on access to the separately retained inputs and recorded software environment.

The following commands are documented in the package README and the product README. Run them from an authorised repository copy with the recorded dependencies available; `EXTERNAL_PACKAGE` denotes the external evaluation package directory shown in Figure 7. The compact verifier uses NumPy and SciPy to regenerate Tables 8, 9, 10 and 11 from retained compact evidence. It invokes the existing joint-confirmation verifier and independently recomputes the other paired summaries; it does not rerun the evaluator or task-level validation. The separately pinned three-trace evidence is read from its recorded Git object.

```
python docs/dissertation/examiner_revision_2026-09-11/document/verify_compact_
→ _tables.py
streamlit run src/traffictwin/ui/app.py
traffictwin integration tos validate "$EXTERNAL_PACKAGE"
```

The first command performs compact arithmetic, the second starts the local platform, and the third inspects an external package through the adapter. Each was executed once in a scratch copy; the command receipt records source identities, exit statuses, the local-platform health response and its controlled shutdown. These checks launched no research workload. Full reproduction additionally requires the retained raw arrays, actor and traces.